Continuum: Concretely Efficient Asynchronous Dynamic MPC with Guaranteed Output Delivery

Anonymous Author(s)

Abstract

Traditional secure multiparty computation (MPC) protocols assume a fixed set of participants throughout the computation. Dynamic MPC (DMPC) relaxes this assumption by permitting parties to join or leave during the process. However, practical deployment of DMPC in Internet settings favors tolerating asynchrony and achieving guaranteed output delivery (GOD). Recently, AD-MPC [CCS 2025] introduced the first asynchronous DMPC (ADMPC) achieving this goal, but its reliance on per-epoch preprocessing and masking randomness incurs substantial concrete overhead. In this paper, we present Continuum, a concretely efficient ADMPC protocol with GOD. We introduce an aggregated hand-off protocol that ensures transfer consistency across epochs while eliminating per-epoch masking randomness, and a batch multiplication protocol based on the BGW paradigm that defers degree reduction to the hand-off phase and aggregates verification for all multiplications into succinct proofs. Together, these protocols enable a non-interactive computation phase with interaction concentrated in an efficient hand-off phase, overcoming the primary concrete inefficiencies of prior ADMPC designs. We implement Continuum and evaluate its performance on up to 16 geographically distributed servers. In experiments on 16 servers with circuits of depth 6 and width 100, Continuum reduces linear-gate transfer latency from 39.64s to 7.25s (82% reduction) and multiplication gate evaluation time from 160.96s to 20.18s (87% reduction), compared with AD-MPC, while preserving strong security guarantees.

CCS Concepts

• **Security and privacy** → **Cryptography**.

Keywords

Secure multiparty computation; Asynchrony; Guaranteed output delivery

ACM Reference Format:
Anonymous Author(s). 2026. Continuum: Concretely Efficient Asynchronous Dynamic MPC with Guaranteed Output Delivery. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26), November 15–19, 2026, Hague, Netherlands.* ACM, New York, NY, USA, 18 pages. https://doi.org/XXXXXXX.XXXXXXX

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CCS '26, Hague, Netherlands.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2025/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Secure multiparty computation (MPC) [8, 34, 45] enables a set of mutually distrustful participants to jointly compute a function while keeping their respective inputs private. However, traditional MPC protocols assume a static set of participants who remain online throughout the entire computation, rendering them unsuitable for dynamic environments where servers may intermittently go offline due to network churn. To address this limitation, recent work has introduced *Dynamic MPC* (DMPC) [18], which partitions the computation into sequential epochs, each handled by a distinct set of participants, termed a *committee*. In a typical DMPC protocol, an epoch consists of two phases: a *computation phase*, during which the current committee evaluates its assigned circuit segment, and a *hand-off phase*, where the resulting secret-shared state is securely transferred to the next committee. DMPC allows participants to dynamically join or leave across epochs, a crucial feature for long-running computations involving complex circuits, such as privacy-preserving machine learning [40], anonymous voting systems [5], and private smart contracts [4, 6].

DMPC naturally aligns with MPC-as-a-Service (MPCaaS), where the set of available servers changes over time. MPCaaS [5, 42] allows resource-constrained clients to outsource privacy-preserving computations to distributed servers, providing a practical deployment setting for DMPC. However, many MPCaaS deployments are desired at the Internet [42], where the message delay can be very large or even unpredictable. To maintain service reliability in such environments, it is essential to achieve *Guaranteed Output Delivery* (GOD), ensuring that all honest parties eventually obtain the correct output despite network instability and adversarial behavior. Therefore, reliable MPCaaS systems necessitate asynchronous DMPC (ADMPC)[1] protocols capable of achieving GOD. However, although numerous DMPC protocols have been proposed for synchronous networks [10, 13, 16–18, 28, 32, 39, 43], their inherent reliance on strong synchrony assumptions precludes them from achieving GOD in asynchronous networks. This gap necessitates the development of concretely efficient ADMPC protocols that achieve GOD.

1.1 Obstacles to Concretely Efficient ADMPC with GOD

To achieve GOD in the asynchronous dynamic setting, an ADMPC protocol requires that every epoch executes correctly and eventually terminates, so that the overall evaluation can keep progressing and the final output is eventually delivered. This GOD guarantee spans both phases of each epoch. During the computation phase, the active committee must correctly evaluate its assigned circuit segment over secret-shared inputs. During the hand-off phase, the resulting secret-shared state must be transferred consistently to the next committee, ensuring that subsequent evaluation proceeds on

[1] ADMPC denotes a class of protocols. AD-MPC [46] refers to the specific instantiation of ADMPC.

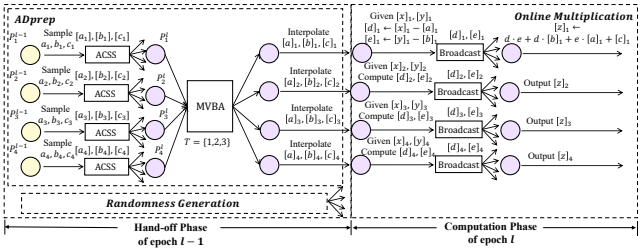
the same underlying values. However, achieving concrete efficiency in an ADMPC protocol with GOD introduces significant challenges to each phase.

Obstacles in the computation phase. Since linear gates are evaluated locally on secret shares, the main challenge during the computation phase concerns the evaluation of multiplication gates. In asynchronous MPC (AMPC) protocols, multiplication gates are typically evaluated using one of two paradigms: the offline/online paradigm based on Beaver triples [3, 19–21, 35, 42] or the BGW paradigm based on direct share multiplication [36, 44]. However, when these paradigms are applied in the asynchronous dynamic setting, both encounter inherent obstacles that significantly impair concrete efficiency.

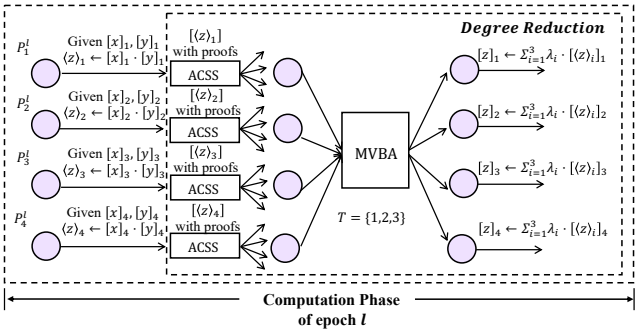
Obstacle #1: Loss of preprocessing amortization in the offline/online paradigm for ADMPC. Existing ADMPC research has primarily leveraged the offline/online paradigm. AD-MPC [46] is the first ADMPC protocol to achieve GOD; in this construction, the active committee runs an asynchronous dynamic preprocessing protocol ADprep to generate Beaver triples for the next epoch, which are then consumed during the computation phase to evaluate multiplication gates, as illustrated in Figure 1(a). Given correct Beaver triples, online multiplication correctness follows directly from the robustness of Shamir secret sharing.

In static AMPC, the paradigm is highly effective because the server set is fixed, allowing parties to run one global preprocessing in advance to generate all required Beaver triples and distribute them to the servers, leaving the online evaluation lightweight. In ADMPC, however, committees are formed on the fly as the evaluation progresses; thus, one cannot run preprocessing once and distribute triples ahead of time for future committees that have not yet been formed. Consequently, ADprep must be invoked separately for each epoch, in lockstep with circuit evaluation. Each invocation incurs $O(\kappa n^2 \log n \cdot M + \kappa n^3)$ communication (where κ denotes the security parameter) for M multiplication gates in that epoch, while the subsequent online evaluation requires $O(\kappa n \cdot M)$ communication. For a circuit whose multiplication gates are distributed across d epochs, this results in $O(d)$ independent preprocessing, compared to the static setting which requires only a single global preprocessing run in advance for all triples, eliminating the amortization benefit that underlies the concrete efficiency of the offline/online paradigm.

Obstacle #2: Verification overhead and absence of hand-off in the BGW paradigm for ADMPC. To mitigate the overhead of epoch-by-epoch preprocessing, one alternative is to evaluate multiplication gates directly on secret-shared inputs using the BGW paradigm [8]. Recent AMPC protocols [36, 44] have adapted BGW-style multiplication to asynchronous networks. In these protocols, parties locally multiply their shares to obtain a degree-$2t$ sharing of the product, and then invoke an asynchronous verifiable degree-reduction procedure to interpolate a standard degree-t sharing. Since BGW-style multiplication avoids the need for per-epoch preprocessing, it can be executed directly during the computation phase, as illustrated in Figure 1(b). Compared to the offline/online paradigm, the BGW paradigm offers concrete communication improvements in the dynamic setting. Specifically, each dealer invokes only a single ACSS instance augmented with a constant-size proof,



(a) Offline/online paradigm



(b) BGW paradigm

Figure 1: Two paradigms for evaluating multiplication gates in the asynchronous dynamic setting. P_i^l denotes the i-th server of the committee $\mathcal{P}^l$. ACSS and MVBA denote asynchronous complete secret sharing [25, 47] and multi-valued validated Byzantine agreement [1, 15], respectively.

whereas the offline/online paradigm requires three ACSS instances per dealer for Beaver triple distribution, in addition to further online communication for consuming the triples.

However, BGW-style multiplication still incurs substantial concrete overhead in the computation phase, as the degree-reduction procedure requires explicit verification proofs. In Dumbo-MPC [44], the current state-of-the-art BGW-style AMPC, degree reduction relies on broadcasting product-relation proofs for the reshared products, resulting in total proof size that grows linearly with the number of multiplication gates M. Although the overall communication complexity remains quadratic $O(\kappa n^2 \cdot M + \kappa n^3)$, this linear proof overhead becomes significant at scale. Moreover, existing BGW-style AMPC protocols are designed for the static setting and lack a compatible hand-off mechanism to transfer the resulting degree-t secret-shared state consistently across epochs.

Obstacles in the hand-off phase. The hand-off phase must ensure transfer consistency: the incoming committee $\mathcal{P}^{l+1}$ must interpolate a fresh degree-t sharing $[s]^{l+1}$ of the same secret s held by the current committee $\mathcal{P}^l$, preserving the underlying values for continued evaluation. Most existing AMPC protocols are designed for the static setting and therefore provide no mechanism to transfer secret-shared state consistently across epochs. AD-MPC [46] introduces the first mechanism to guarantee this consistency in the asynchronous dynamic setting. However, achieving this guarantee

incurs substantial concrete overhead, making the hand-off phase a primary bottleneck to concretely efficient ADMPC with GOD.

Obstacle #3: Masking randomness overhead in AD-MPC. To guarantee transfer consistency, AD-MPC [46] relies on a mask-and-reconstruct strategy, as illustrated in Figure 2. In epoch $l-1$, committee $\mathcal{P}^{l-1}$ distributes fresh masking randomness $[r]^l$ to committee $\mathcal{P}^l$. Upon receiving this randomness, $\mathcal{P}^l$ invokes the asynchronous dynamic transfer protocol ADtrans to mask each sharing $[s]^l$ and broadcast the masked values with correctness proofs, enabling committee $\mathcal{P}^{l+1}$ to interpolate fresh degree-t sharings $[s]^{l+1}$. The primary inefficiency of this strategy stems from the recurring requirement for masking randomness in every epoch. To transfer W sharings, the Randomness Generation step incurs $O(\kappa n \log n \cdot W + \kappa n^3)$ communication, followed by the ADtrans invocation which incurs $O(\kappa n^2 \log n \cdot W + \kappa n^3)$ communication. Although the communication cost associated with randomness generation is asymptotically smaller than that of ADtrans, it is unavoidable and recurs every epoch, further increasing concrete bandwidth overhead.

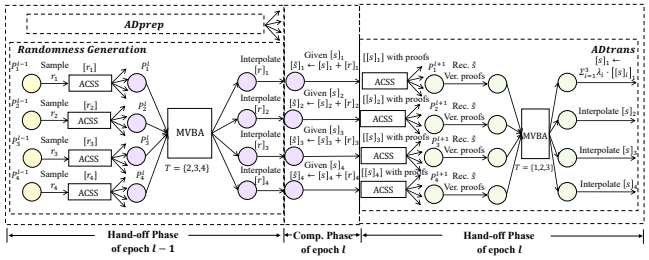


Figure 2: Mask-and-reconstruct strategy in AD-MPC. The strategy requires two consecutive steps for state transfer: first, committee $\mathcal{P}^{l-1}$ distributes fresh masking randomness $[r]^l$ to committee $\mathcal{P}^l$; subsequently, $\mathcal{P}^l$ invokes the ADtrans protocol to mask sharing $[s]^l$ with $[r]^l$ and broadcasts the masked values, enabling $\mathcal{P}^{l+1}$ to interpolate fresh degree-t sharings $[s]^{l+1}$.

1.2 Our Contributions

In this work, we present Continuum, a concretely efficient asynchronous dynamic MPC protocol achieving guaranteed output delivery. Continuum introduces a novel aggregated hand-off protocol AggTrans, and a BGW-compatible hand-off mechanism BatchMul, which achieve transfer consistency and multiplication correctness across epochs, respectively. Together, these protocols enable a non-interactive computation phase with interaction concentrated in an efficient hand-off phase, yielding a per-epoch communication complexity of $O(\kappa n^2 \cdot W + \kappa n^3)$ for circuit width W. In sum, our contributions are:

- **Aggregated hand-off protocol eliminating masking randomness.** We present AggTrans, a novel aggregated hand-off protocol that guarantees transfer consistency while eliminating masking randomness overhead. We first introduce an augmented KZG polynomial commitments with aggregation interfaces that compresses evaluation proofs for multiple polynomials at a common point into a single constant-sized proof. A key feature of our construction is *public verifiability*: any external verifier can check the aggregated evaluation proof using only public commitments, without accessing secret shares. Using this augmented KZG commitment, consistency is ensured through publicly verifiable checks of the handed-off shares against prior-epoch commitments, combined with agreement on a common subset of valid senders, thereby eliminating the dependency for per-epoch masking randomness.
- **BGW-based batch multiplication with deferred degree reduction.** We design BatchMul, a batch multiplication protocol based on the BGW paradigm that eliminates the need for per-epoch preprocessing. To address the imcompatibility of prior BGW-style AMPC protocols with dynamic settings, we design a BGW-compatible hand-off mechanism by extending AggTrans. The core idea is to defer verifiable degree reduction to the hand-off phase: parties can locally multiply their shares without communication, enabling a non-interactive computation phase that produces degree-$2t$ sharings of products. The committee then reshares these intermediate sharings via the hand-off phase while simultaneously proving the required product relations. By employing an inner-product argument, we aggregate verification checks for all M multiplications in an epoch into a single $O(\log M)$-size proof. Since linear-gate transfer via AggTrans and multiplication verification via BatchMul proceed in parallel during the same hand-off phase, the overall epoch latency is is significantly compressed. Moreover, our approach of proving product relations of Shamir secret shares over KZG commitments could be of independent interest, such as making offline/online AMPCaaS with fixed participants more efficient [44].
- **Formal proof and prototype implementation.** We provide a formal security analysis of Continuum, demonstrating resilience against adversaries corrupting up to $t < n/3$ servers, alongside a prototype implementation evaluated on up to 16 distributed servers. As a highlight of our experimental results, Table 1 compares AD-MPC and Continuum under a circuit of width 100 and depth 6, where the circuit is entirely composed of either linear gates or multiplication gates, with a committee size of 16. Our results show that (i) the aggregated hand-off protocol AggTrans reduces the linear-gate transfer latency from 39.64s to 7.25s (an 82% reduction), and (ii) the batch multiplication protocol BatchMul reduces the multiplication gate evaluation time from 160.96s to 20.18s (an 87% reduction). Our implementation is open-sourced at https://anonymous.4open.science/r/ad-mpc-pp-C456.

2 Related Work

DMPC in the synchronous setting was pioneered by Fluid MPC [18], which introduced the notion of maximal fluidity, allowing a party to exit the computation immediately after completing a single round of sending and receiving messages. Subsequent synchronous DMPC protocols preserved maximal fluidity while achieving additional

Table 1: Comparison of AD-MPC and Continuum on a width-100, depth-6 circuit with committee size $n = 16$. The circuit consists entirely of either linear gates or multiplication gates.

Protocols	Linear-gate transfer	Multiplication-gate evaluation
AD-MPC [46]	39.64s	160.96s
Continuum	7.25s (↓ 82%)	20.18s (↓ 87%)

properties, such as tolerating a dishonest majority [10, 43] or attaining linear communication complexity [10, 11]. However, these protocols provide only Security with Abort (SwA), meaning that honest parties can detect misbehavior but are not guaranteed to obtain the output. A line of work has further explored achieving GOD in the synchronous setting. DDG+23 [28] attains GOD under maximal fluidity, offering one protocol with perfect security for $t < n/3$ and another with computational security for $t < n/2$. DKLN25 [29] strengthens these results by showing that GOD can also be achieved with statistical security for $t < n/2$. Despite these advances, all such protocols fundamentally rely on synchrony and therefore cannot tolerate the arbitrary message delays of asynchronous networks. The first attempt to bridge this gap is AD-MPC [46], which achieves GOD in the asynchronous setting without assuming maximal fluidity. While AD-MPC demonstrates the feasibility of ADMPC with GOD, its concrete efficiency motivates further improvements.

Another related line of work is the YOSO (You Only Speak Once) model for MPC [2, 9, 16, 17, 22, 23, 30, 32, 39, 41], which also seeks maximal fluidity by ensuring that each committee speaks only once before being replaced. This is achieved by hiding a committee's identity until after its single broadcast, and has been studied in both synchronous and asynchronous settings. Compared to DMPC, YOSO MPC operates under a different threat model and incurs additional overhead from the role assignment protocol required to anonymously select committee members. Due to these fundamental differences, we do not consider YOSO MPC directly comparable to our setting.

3 Model and Preliminaries

In this section, we formalize our system model and threat assumptions, defines the security goals of ADMPC, and introduces the necessary asynchronous building blocks used in our construction.

3.1 Model and Design Goals

Participants and Trusted Setup. We denote by $\mathcal{S} = \{P_1, \ldots, P_N\}$ a finite set of N participating servers. We assume the existence of a public-key infrastructure (PKI), where each server $P_i \in \mathcal{S}$ holds a public-private key pair (pk_i, sk_i), and all public keys pk_i are publicly known. In addition, we assume that the common reference string (CRS) $(g, g^{\alpha}, \cdots, g^{\alpha^t}, h, h^{\alpha}, \cdots, h^{\alpha^t}) \in \mathbb{G}^{2t+2}$ of KZG polynomial commitment [38] is honestly generated and publicly available. This CRS is generated once during the MPCaaS system deployment via an asynchronous distributed protocol [26]. The protocol is executed by an initial committee to support a fixed maximum degree, which corresponds to the system's security threshold. The resulting CRS is reusable across arbitrary circuit evaluations within this degree bound.

Asynchrony. We assume an asynchronous network, featuring authenticated point-to-point channels among all parties. Under this asynchronous model, there are no timing constraints, and messages may experience arbitrary delays; however, all messages sent between honest parties must eventually be delivered [7].

Dynamism. We consider an ADMPC protocol consisting of three consecutive stages: input, execution, and output. During the input stage, clients submit their private inputs and computational tasks to an initial committee $\mathcal{P}^1 \subseteq \mathcal{S}$. The execution stage involves evaluating the submitted tasks, represented as a layered arithmetic circuit C with depth d, partitioned into epochs $\{C^l\}_{l \in [d]}$. For each epoch $l \in [d]$, the active committee $\mathcal{P}^l \subset \mathcal{S}$ first evaluates the epoch C^l in the computation phase, and subsequently transfers the resulting secret-shared outputs to the next committee $\mathcal{P}^{l+1}$ during the hand-off phase. Finally, during the output stage, the last committee $\mathcal{P}^d$ delivers the final computation results to the clients.

Committee Configuration. Each committee consists of n servers, selected from the global server set $\mathcal{S}$ according to a committee selection procedure (e.g., Algorand [33]). We impose no constraints on committee overlap; a server may participate in multiple epochs. Once an epoch is completed, each server in the active committee erases its internal state, rendering post-execution corruption ineffective.

Threat Model. We follow the R-adaptive adversary model as in AD-MPC [46], wherein a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ may adaptively corrupt up to $t < n/3$ servers at the beginning of each epoch C^l with committee $\mathcal{P}^l$. Once a server is corrupted, $\mathcal{A}$ obtains its entire past state and may impersonate it in future communications. If a server P_i that previously participated in epoch C^m $(m < l)$ is corrupted during C^l, the corruption is permitted only if it does not increase the number of corrupted servers in C^m beyond the threshold t. In addition, the adversary may corrupt up to c out of k clients submitting inputs during the input stage. For simplicity, we assume $n = k$ and $t = c$.

Design Goals. We aim to construct a concretely efficient asynchronous dynamic MPC protocol that achieves guaranteed output delivery against an R-adaptive adversary. Formally, we define our goal as follows:

Definition 3.1 (Asynchronous Dynamic MPC with Guaranteed Output Delivery). An asynchronous dynamic MPC protocol for a layered arithmetic circuit with depth d and committees of size $n = 3t + 1$ achieves guaranteed output delivery if, under the presence of a R-adaptive PPT adversary $\mathcal{A}$ corrupting at most t parties per epoch, it satisfies the following properties:
Privacy: For each epoch $l \in [d]$, the adversary $\mathcal{A}$ learns nothing beyond what is revealed by the corrupted servers and clients.
Guaranteed Output Delivery (Robustness): For each epoch $l \in [d]$, all honest servers in $\mathcal{P}^l$ eventually output the evaluation of epoch C^l on at least $n - t$ distinct inputs received from $\mathcal{P}^{l-1}$. This robustness is further refined into two sub-properties:

- *Transfer Consistency.* After the computation phase of epoch C^{l-1}, for every degree-t sharing $[s]^{l-1}$ held by honest servers in $\mathcal{P}^{l-1}$, the hand-off phase ensures that honest servers in

$\mathcal{P}^l$ interpolate a fresh degree-t sharing $[s]^l$ of the same underlying secret.

- *Multiplication Correctness.* For each multiplication gate in C^{l-1} with input degree-t sharings $[a]^{l-1}$ and $[b]^{l-1}$ held by honest servers in $\mathcal{P}^{l-1}$, the servers locally compute a degree-$2t$ intermediate sharing $\langle c \rangle^{l-1}$. The hand-off phase ensures that honest servers in $\mathcal{P}^l$ interpolate a fresh degree-t sharing $[c]^l$ that encodes the exact product $c = a \cdot b$.

3.2 Preliminaries

Batch Asynchronous Complete Secret Sharing. A batch asynchronous complete secret sharing (BACSS) protocol consists of two phases: Sharing and Reconstruction. During the sharing phase, a dealer shares B secrets $(s_1, \cdots, s_B)$ using Share. During the reconstruction phase, parties use Rec to recover these secrets. BACSS guarantees the following properties under asynchronous conditions:

- *Correctness:* If the dealer is honest, each honest party P_i will eventually receive shares $([s_1]_i, \cdots, [s_B]_i)$ during the sharing phase.
- *Secrecy:* If the dealer is honest and no honest parties have begun the reconstruction phase, then a computationally bounded adversary corrupting up to t parties gains no information about s_k for any $k \in [B]$, except with negligible probability.
- *Completeness:* If any honest party completes the sharing phase, then for $k \in [B]$, there exists a degree-t polynomial $f_k(\cdot)$ over $\mathbb{Z}_q[x]$ such that $f_k(0) = s'_k$, and all honest parties P_i will eventually receive their shares $[s'_k]_i = f_k(i)$. Additionally, when the dealer is honest, $s'_k = s_k$.

One of state-of-the-art BACSS protocols is hbACSS [47], which provides a modular framework that can be instantiated with various polynomial commitment schemes. In AD-MPC [46], hbACSS is instantiated with hbPolyCom [47], resulting in a communication complexity of $O(\kappa n \log n \cdot B)$, while in our setting we use KZG polynomial commitments [38], achieving $O(\kappa n \cdot B)$ for sharing B secrets.

Multi-valued Validated Byzantine Agreement. A Multi-valued Validated Byzantine Agreement (MVBA) protocol [15] allows a set of parties, where each party provides an input value, to eventually agree on the same value that satisfies a predefined external predicate $P(m) : \{0,1\}^{|m|} \to \{0,1\}$, which is known to all parties. The MVBA protocol satisfies the following properties [37]:

- *Termination:* If every honest party inputs with an externally valid value, then every honest party outputs a value.
- *External-Validity*: If an honest party outputs a value m, then $P(m) = 1$.
- *Agreement:* All honest parties that terminate output the same value.

We employ the MVBA protocol from [24, 27, 44] as a building block, which has a communication complexity of $O(\kappa n^3)$.

4 Asynchronous Dynamic MPC

We now present Continuum, our concretely efficient ADMPC protocol with GOD. We begin by outlining its overall structure. The protocol operates over a layered arithmetic circuit C of depth d, where each layer corresponds to an epoch. Let C^l denote the l-th epoch, characterized by its width w^l, which consists of λ^l linear gates and μ^l multiplication gates. Each epoch is evaluated by a committee $\mathcal{P}^l$ consisting of $n = 3t + 1$ servers, of which at most t may be corrupted. We assume an initial client set responsible for providing secret inputs. The execution of Continuum proceeds in three stages:

- **Input Stage:** Clients distribute their private inputs to the first committee $\mathcal{P}^1$ using BACSS.
- **Execution Stage.** During each epoch $l \in [d]$, committee $\mathcal{P}^l$ evaluates the corresponding epoch C^l through two sequential phases, as illustrated in Figure 3. Throughout epoch l, each wire value s is represented by a degree-t sharing $[s]$ together with a public polynomial commitment C_s[2]. Each server P_i holds the local share $[s]_i$ along with an evaluation proof $w_{s,i}$ proving that $[s]_i$ lies on the degree-t polynomial committed to by C_s.
  Computation Phase. Each server P_i locally evaluates all w gates without interaction. For the k-th linear gate ($k \in [\lambda]$), with input wires x_k and y_k and output wire z_k, P_i computes the output share
  $$[z_k]_i \leftarrow \mathsf{LinEval}([x_k]_i, [y_k]_i).$$
  The associated commitment C_{z_k} and evaluation proof $w_{z_k,i}$ are updated linearly from the input commitments C_{x_k}, C_{y_k} and and the corresponding proofs $w_{x_k,i}$, $w_{y_k,i}$, respectively, following the homomorphic properties of the commitment scheme.
  For the k-th multiplication gate ($k \in [\mu]$), with input wires a_k, b_k and output wire c_k, each server P_i locally multiplies its input shares to obtain an intermediate degree-$2t$ share
  $$\langle c_k \rangle_i \leftarrow \mathsf{MulEval}([a_k]_i, [b_k]_i),$$
  yielding a degree-$2t$ sharing $\langle c_k \rangle$.
  Hand-off Phase. The committee $\mathcal{P}^l$ transfers the secret-shared state to the next committee $\mathcal{P}^{l+1}$. Specifically, $\mathcal{P}^l$ invokes AggTrans to transfer degree-t sharings of linear gate outputs to ensure transfer consistency. Concurrently, $\mathcal{P}^l$ invokes BatchMul to reshare and reduce the degree of intermediate sharings, guaranteeing multiplication correctness.
- **Output Stage.** After the final epoch, clients receive shares of the outputs from committee $\mathcal{P}^d$ via the hand-off phase. The clients then broadcast these shares and reconstruct the final results, leveraging the robustness of Shamir secret sharing.

Security and Overall Complexity. Full proofs and detailed complexity analysis for Continuum appear in Appendix D.

5 Aggregated Hand-off Protocol

In this section, we present AggTrans, an aggregated hand-off protocol designed to ensure transfer consistency while eliminating the overhead of per-epoch masking randomness. We begin by providing a high-level intuition of our approach and explaining how it overcomes the limitations of the prior mask-and-reconstruct strategy.

[2]For readability, within epoch l we omit the superscript l on local quantities (e.g., $[s]^l$, C_s^l, $w_{s,i}^l$) when the epoch is clear from context.

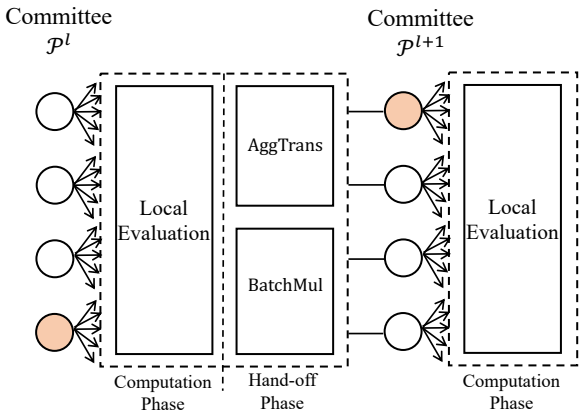


Figure 3: Illustration of the execution stage in Continuum**, including local evaluation, hand-off via** AggTrans **and** BatchMul**. Orange nodes indicate corrupted parties.**

To support this design, we introduce an augmented KZG polynomial commitment scheme with aggregation interfaces. Finally, we detail the formal phases of the AggTrans protocol.

5.1 Motivation: Eliminating the Masking Randomness Bottleneck

The hand-off phase must ensure transfer consistency across epochs. The core idea of AD-MPC's hand-off phase is to enable each server $P_i^l \in \mathcal{P}^l$ to reshare its local share $[s]_i^l$, which is associated with a public polynomial commitment C_s^l. At a high level, P_i^l reshares its local share by distributing a new degree-t sharing $[[s]_i^l]^{l+1}$ to the next committee $\mathcal{P}^{l+1}$, such that the constant term of this new sharing is $[s]_i^l$. Honest servers in $\mathcal{P}^{l+1}$ then collect these resharings, agree on a common subset of dealers via consensus, and interpolate a fresh degree-t sharing $[s]^{l+1}$ encoding the same underlying secret. However, this high-level flow is vulnerable to inconsistency attacks: a malicious dealer may distribute a deviant resharing $[[s']_i^l]^{l+1}$. If such inconsistent resharings are included in the interpolation, honest servers in $\mathcal{P}^{l+1}$ will interpolate an incorrect sharing $[s']^{l+1}$ encoding $s' \neq s$. To defend against such attacks, AD-MPC introduces a mask-and-reconstruct strategy, which involves two consecutive steps: first, committee $\mathcal{P}^{l-1}$ distributes fresh masking randomness to committee $\mathcal{P}^l$, and subsequently, $\mathcal{P}^l$ invokes the ADtrans protocol to mask the sharing with masking randomness and broadcast the masked values with correctness proofs. This strategy creates a recurring masking-randomness pipeline that operates independently of circuit evaluation, incurring substantial concrete overhead. This necessity defines a fundamental *anchoring condition*: every accepted resharing $[[s]_i^l]^{l+1}$ must be verifiably anchored to the prior commitment C_s^l, such that its constant term $[s]_i^l$ is a valid evaluation at the dealer's index i.

Our Approach. We achieve this condition by introducing an augmented KZG commitment scheme with an aggregation interface. Instead of relying on masking, each dealer provides a single constant-size, publicly verifiable proof that certifies their reshared constant terms are valid evaluations of the commitments they broadcast. This construction enables any server in the next committee $\mathcal{P}^{l+1}$ to independently verify the internal consistency of each dealer's resharing solely from the public transcript, without any access to original secret shares. However, these per-dealer public verifiability checks alone are insufficient to satisfy the anchoring condition. A malicious dealer could broadcast an arbitrary commitment $C_{s'}^l$ and provide a valid proof for a deviant share $[s']_i^l \neq [s]_i^l$, bypassing the above public verifiability checks. To defend against such attacks and ensure all accepted sharings are anchored to the original global commitment C_s^l, the next committee $\mathcal{P}^{l+1}$ invokes consensus to agree on a common subset of dealers who provide matching commitments that are identical to the globally agreed-upon commitments from the previous epoch. Interpolation over this agreed subset then interpolates fresh degree-t sharings of the original secrets, achieving transfer consistency without any masking randomness.

5.2 Aggregation Interfaces over KZG Polynomial Commitments

To eliminate the substantial overhead of per-epoch masking randomness, we augment the standard KZG polynomial commitment scheme [38] with aggregation interfaces. As detailed in Figure 4, these interfaces enable non-shareholders to indenpendently verify that reshared constant term are valid evaluations of committed polynomials, providing the cryptographic foundation to satisfy the anchoring condition.

The standard VerifyEval function restricts verification to shareholders, as it requires the secret share $f(i)$ and randomness $\hat{f}(i)$ as inputs. To achieve public verifiability, we introduce the PubVerEval interface, where the prover provides a Pedersen-style commitment $g^{f(i)}h^{\hat{f}(i)}$ instead of the secret share and randomness. This allows non-shareholders to verify evaluation proofs while maintaining information-theoretic hiding of the underlying share $f(i)$. To further reduce concrete overhead, we incorporate aggregation interfaces inspired by [12], tailored for B degree-t polynomials evaluated at a single common point i. The functions AggPubProEval and AggPubVerEval enable a dealer to generate a single constant-size aggregated evaluation proof for all B committed polynomials.

Security Properties. Our proposed KZG scheme with aggregation interfaces satisfies *correctness*, *public verifiability*, *evaluation binding*, *polynomial binding*, *aggregation binding*, and *hiding*, under the hardness assumptions of the discrete logarithm and t-strong Diffie–Hellman problems. We provide formal definitions and proofs of these properties in Appendix A.

5.3 Construction of the AggTrans Protocol

We now present the detailed construction of AggTrans, which fulfills the anchoring condition to ensure transfer consistency without per-epoch masking randomness. The procedure is summarized in Algorithm 1 and organized into four phases:

(1) *Aggregated resharing (lines 101–107).* Each server $P_i^l \in \mathcal{P}^l$ reshares its shares $\{[s_k]_i^l\}_{k\in[B]}$ by distributing new degree-t polynomials $f_{ki}^{l+1}(x)$ whose $f_{ki}^{l+1}(0) = [s_k]_i^l$. Crucially, P_i^l leverages the AggPubProEval interface to generate a succinct evaluation proof $(T_{i,0}^{l+1}, W_{i,0}^{l+1})$ for all B evaluations at

KZG Polynomial Commitment with Aggregation

Setup$(1^\kappa, t)$: generate bilinear pairing group $\mathcal{G} = (e, \mathbb{G}, \mathbb{G}_T)$ and randomly sample $\alpha, \tau \in \mathbb{Z}_q$. Let g be random generator of $\mathbb{G}$, set $h = g^\tau$ and return $SP = \{\mathcal{G}, \{g^{\alpha^i}, h^{\alpha^i}\}_{i=0}^t\}$.

PolyCom$(SP, f(\cdot))$: sample random degree- t polynomial $\hat{f}(\cdot)$, and compute $C = g^{f(\alpha)} h^{\hat{f}(\alpha)}$.

ProveEval$(SP, i, f(\cdot), \hat{f}(\cdot))$: compute $w_i = g^{q_i(\alpha)} h^{\hat{q}_i(\alpha)}$, where $q_i(x) = \frac{f(x)-f(i)}{x-i}$ and $\hat{q}_i(x) = \frac{\hat{f}(x)-\hat{f}(i)}{x-i}$, output $(i, f(i), \hat{f}(i), w_i)$.

VerifyEval$(SP, C, i, f(i), \hat{f}(i), w_i)$: output 1 if $e(C/(g^{f(i)} h^{\hat{f}(i)}), g) = e(w_i, g^\alpha / g^i)$, output 0 otherwise.

Below are aggregation interfaces

PubVerEval$(SP, C, i, g^{f(i)} h^{\hat{f}(i)}, w_i)$: output 1 if $e(C/(g^{f(i)} h^{\hat{f}(i)}), g) = e(w_i, g^\alpha / g^i)$, output 0 otherwise.

AggPubProEval$(SP, i, \{f_j(\cdot), \hat{f}_j(\cdot), C_j\}_{j\in B})$: compute challenge $\gamma = \mathcal{H}(SP, i, \{C_j\}_{j\in B})$. Compute aggregated polynomials $h_i(x) = \sum_{j\in B} \gamma^{j-1} \cdot \frac{f_j(x)-f_j(i)}{x-i}$, $\hat{h}_i(x) = \sum_{j\in B} \gamma^{j-1} \cdot \frac{\hat{f}_j(x)-\hat{f}_j(i)}{x-i}$, and $W_i = g^{h_i(\alpha)} h^{\hat{h}_i(\alpha)}$. Aggregate $T_i = \prod_{j\in B} (g^{f_j(i)} h^{\hat{f}_j(i)})^{\gamma^{j-1}}$. Output (i, T_i, W_i).

AggPubVerEval$(SP, i, T_i, W_i, \{C_j\}_{j\in B})$: recompute the challenge γ and aggregate commitments $C = \prod_{j\in B} C_j^{\gamma^{j-1}}$. Output 1 if the aggregated proof passes the pairing check $e(C/T_i, g) = e(W_i, g^\alpha / g^i)$, and output 0 otherwise.

AggOpenEval$(SP, i, T_i, \{f_j(i), \hat{f}_j(i), C_j\}_{j\in B})$: recompute the challenge γ. If $T_i = \prod_{j\in B} (g^{f_j(i)} h^{\hat{f}_j(i)})^{\gamma^{j-1}}$, output 1, otherwise output 0.

Figure 4: Our augmented KZG polynomial commitment with aggregation interfaces. We describe its non-interactive variant using the Fiat–Shamir heuristic [31] in the random oracle model of the cryptographic hash function $\mathcal{H}$.

point 0. This proof, alongside the prior-epoch commitments $\{C_k^l\}_{k\in[B]}$ and the compressed prior-epoch evaluation proof W_i^l, enables the next committee to satisfy the anchoring condition and eliminates the need for masking randomness.

(2) *Verification of anchoring (lines 201–206).* This phase utilizes the public verifiability of our augmented KZG scheme to locally verify the consistency of each dealer's resharing relative to the prior epoch. Each $P_j^{l+1} \in \mathcal{P}^{l+1}$ performs the following checks:
- **Reshared share validity:** For every $k \in [B]$, VerifyEval ensures $[[s_k]_i^l]_j^{l+1}$ is a correct evaluation of the new degree-t polynomial $f_{ki}^{l+1}(x)$ at point j.
- **Internal consistency:** AggPubVerEval verifies the constant terms $\{f_{ki}^{l+1}(0)\}_{k\in[B]}$ match the aggregate commitment $T_{i,0}^{l+1}$.
- **Local anchoring to prior epoch:** Another AggPubVerEval confirms $T_{i,0}^{l+1}$ is consistent with the prior-epoch commitments $\{C_k^l\}_{k\in[B]}$.

These publicly verifiable checks allow $\mathcal{P}^{l+1}$ to filter out inconsistent or malformed resharings before the agreement phase.

(3) *Agreement on matching commitments (lines 207–209).* While local verification filters out malformed resharings, it cannot individually guarantee that a dealer's commitments match the globally agreed-upon commitments from the previous epoch. A malicious dealer could provide shares that are consistent with its own commitments but diverge from the true global commitments. To prevent this, each P_j^{l+1} identifies a candidate subset $T_j' \subseteq T_j$ of size $2t+1$ whose members provide matching polynomial commitments $\{C_k^l\}_{k\in[B]}$ for the entire batch. P_j^{l+1} then invokes MVBA with T_j' as input to reach consensus on a common subset of valid senders T. This step ensures that all servers in the new committee agree on the same set of resharings that are correctly anchored to the original secrets.

(4) *Interpolation of new secret shares (lines 210–213).* After consensus is reached on the valid dealer set T, each P_j^{l+1} interpolates its final secret shares $\{[s_k]_j^{l+1}\}_{k\in[B]}$ using Lagrange interpolation over the shares $\{[[s_k]_i^l]_j^{l+1}\}_{i\in T}$. This yields a fresh degree-t sharing of each secret across the new committee $\mathcal{P}^{l+1}$, successfully preserving transfer consistency throughout the epoch transition.

Security of AggTrans**.** The security properties of AggTrans, including *termination*, *transfer consistency*, and *secrecy*, are formally stated and proven in Appendix B.

Complexity of AggTrans**.** Executing $\Theta(n)$ BACSS instances in parallel to share B secrets yields $O(\kappa\, n^2 \cdot B)$ communication, while a single MVBA invocation adds $O(\kappa\, n^3)$. Thus, the overall complexity of AggTrans is $O(\kappa\, n^2 \cdot B + \kappa\, n^3)$.

6 Batch Multiplication Evaluation

In this section, we present BatchMul, a specialized variant of AggTrans designed to ensure multiplication correctness. By adapting the BGW paradigm and deferring verifiable degree reduction to the hand-off phase, BatchMul enables a non-interactive computation phase. We further leverage a tailored inner-product argument over augmented KZG commitments to aggregate the verification of product relations for all M multiplication gates into a single $O(\log M)$-size proof. The remainder of this section motivates our design by analyzing these overheads in existing protocols, details our inner-product argument construction, and provides the formal construction of BatchMul.

6.1 Motivation: Overcoming Interaction and Proof Overheads in BGW-style ADMPC

While AD-MPC [46] introduces the first ADMPC protocol based on the offline/online paradigm, its reliance on epoch-coupled preprocessing eliminates the structural amortization benefits typically associated with this paradigm. As an alternative, the BGW paradigm enables evaluating multiplication gates directly on secret

Algorithm 1 Asynchronous dynamic transfer for $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$

Public Inputs: The batch size B, KZG commitments $\{C_k^l\}_{k\in[B]}$, and other system parameters

Private Inputs: P_i^l holds secret shares $\{[s_k]_i^l\}_{k\in[B]}$ with randomness $\{[\hat{s}_k]_i^l\}_{k\in[B]}$, evaluation proofs $\{w_{ki}^l\}_{k\in[B]}$

Public Outputs: New KZG commitments $\{C_k^{l+1}\}_{k\in[B]}$

Private Outputs: P_j^{l+1} obtains $\{[s_k]_j^{l+1}\}_{k\in[B]}$ with randomness $\{[\hat{s}_k]_j^{l+1}\}_{k\in[B]}$ and evaluation proofs $\{w_{kj}^{l+1}\}_{k\in[B]}$

Server P_i^l (in committee $\mathcal{P}^l$):

// 1. Reshare secrets using aggregation interfaces

for $k \in [B]$

$(C_{ki}^{l+1}, \hat{f}_{ki}^{l+1}(\cdot)) \leftarrow \mathsf{PolyCom}(SP, f_{ki}^{l+1}(\cdot))$,

// s.t. $f_{ki}^{l+1}(0) = [s_k]_i^l$, $\hat{f}_{ki}^{l+1}(0) = [\hat{s}_k]_i^l$

for $j \in [n]$

$w_{ki,j}^{l+1} \leftarrow \mathsf{ProveEval}(SP, j, f_{ki}^{l+1}(\cdot), \hat{f}_{ki}^{l+1}(\cdot))$

$(T_{i,0}^{l+1}, W_{i,0}^{l+1}) \leftarrow \mathsf{AggPubProEval}(SP, 0, \{f_{ki}^{l+1}(\cdot), \hat{f}_{ki}^{l+1}(\cdot), C_{ki}^{l+1}\}_{k\in[B]})$

aggregate $W_i^l \leftarrow \prod_{k\in[B]} (w_{k,i}^l)^{\gamma_i^{k-1}}$ with the same challenge $\gamma_i \leftarrow \mathcal{H}(SP, i, \{C_{ki}^{l+1}\}_{k\in[B]})$

invoke $\mathsf{BACSS_share}_i$ as dealer to reshare $\{[s_k]_i^l\}_{k\in[B]}$ using KZG commitments $\{C_{ki}^{l+1}\}_{k\in[B]}$ with broadcasting $(\{C_k^l\}_{k\in[B]}, W_i^l, W_{i,0}^{l+1}, T_{i,0}^{l+1})$

Server P_j^{l+1} (in committee $\mathcal{P}^{l+1}$):

// 2. Locally verify shares and filter invalid dealers

$T_j \leftarrow \emptyset$, $\mathsf{old_shares}_j \leftarrow \emptyset$, $\mathsf{new_shares} \leftarrow \emptyset$

activate $\mathsf{BACSS_share}_i$ instance for every $i \in [n]$

upon receiving $\{C_{k,i}^{l+1}, [[s_k]_i^l]_j^{l+1}, [[\hat{s}_k]_i^l]_j^{l+1}, w_{k,i,j}^{l+1}, C_k^l\}_{k\in B}$ and $(W_i^l, W_{i,0}^{l+1}, T_{i,0}^{l+1})$ from $\mathsf{BACSS_share}_i$

// s.t. $[[s_k]_i^l]_j^{l+1} = f_{ki}^{l+1}(j)$, $[[\hat{s}_k]_i^l]_j^{l+1} = \hat{f}_{ki}^{l+1}(j)$

if $\forall k \in [B]$, it holds that

$$\mathsf{VerifyEval}(SP, C_{ki}^{l+1}, j, [[s_k]_i^l]_j^{l+1}, [[\hat{s}_k]_i^l]_j^{l+1}, w_{ki,j}^{l+1}) = 1$$
$$\wedge\ \mathsf{AggPubVerEval}(SP, 0, T_{i,0}^{l+1}, W_{i,0}^{l+1}, \{C_{ki}^{l+1}\}_{k\in[B]}) = 1$$
$$\wedge\ \mathsf{AggPubVerEval}(SP, i, T_{i,0}^{l+1}, W_i^l, \{C_k^l\}_{k\in[B]}) = 1$$

// where $\{C_k^l\}_{k\in[B]}$ is aggregated with the same γ_i

$\mathsf{old_shares}_j \leftarrow \mathsf{old_shares}_j \cup \{[[s_k]_i^l]_j^{l+1}\}_{k\in[B]}$

$T_j \leftarrow T_j \cup \{i\}$

// 3. Agree on subset with matching commitments

if $|T_j| \geq 2t+1$ **and** $\exists T_j' \subseteq T_j$ with $|T_j'| = 2t+1$ such that all commitments $\{C_k^l\}_{k\in[B]}$ are identical

call MVBA using T_j' as input and T_j as state

// MVBA's predicate checks if $T_j \supseteq$ input

wait for MVBA output T, w.l.o.g, let $T = \{P_1^l, \cdots, P_{2t+1}^l\}$

// 4. Interpolate new secret shares

for $k \in [B]$

$[s_k]_j^{l+1} \leftarrow \Sigma_{i=1}^{t+1} \lambda_i [[s_k]_i^l]_j^{l+1}$

// λ_i is the Lagrange coeff. for eval at 0

// update the corresponding KZG commitments and randomness for the new shares

$\mathsf{new_shares} \leftarrow \mathsf{new_shares} \cup \{[s_k]_j^{l+1}\}_{k\in[B]}$

return $\mathsf{new_shares}$

shares, avoiding the need for a separate preprocessing pipeline. Because both the state-of-the-art BGW-style AMPC protocol Dumbo-MPC [44] and our AggTrans are built upon KZG polynomial commitments, we could construct a BGW-style ADMPC by sequentially employing the multiplication protocol of Dumbo-MPC followed by AggTrans. In this construction, the active committee first invokes Dumbo-MPC's multiplication protocol during the computation phase to yield standard degree-t sharings, and subsequently invokes AggTrans during the hand-off phase to transfer these sharings to the next committee.

However, such a straightforward construction incurs prohibitive concrete overhead. First, the degree-reduction procedure in the computation phase requires broadcasting individual product-relation proofs for each gate. Consequently, the total proof size grows linearly with the number of multiplication gates, incurring substantial overhead as the circuit width increases. Second, and more critically, this design suffers from redundant interaction rounds. To complete the verifiable degree reduction in the computation phase, the committee must invoke multiple instances of BACSS along with an MVBA instance. Similarly, invoking AggTrans in the hand-off phase also requires multiple BACSS instances and a separate MVBA instance to ensure transfer consistency. This sequential execution of two interaction-heavy phases effectively doubles the epoch latency.

Our Approach. We adopt the BGW paradigm while introducing two key technical optimizations to resolve the concrete inefficiencies inherent in the above construction. To eliminate linear proof overhead, we leverage our augmented KZG commitment together with a tailored inner-product argument that compresses proofs for all M reshared products into a single $O(\log M)$-size argument per dealer. To resolve the redundant interaction bottleneck, we redesign the protocol as a variant of AggTrans whose core innovation is to defer the verifiable degree-reduction procedure to the hand-off phase. Specifically, the computation phase becomes non-interactive, as each server in $\mathcal{P}^l$ locally multiplies its degree-t shares to obtain degree-$2t$ shares of the products. In the hand-off phase, each server in $\mathcal{P}^l$ reshares these products to $\mathcal{P}^{l+1}$ by extending the aggregated hand-off protocol to simultaneously satisfy the anchoring condition and prove multiplicative relations. This approach ensures multiplication correctness across epochs with only logarithmic-size proofs and interaction concentrated in the hand-off phase per epoch.

6.2 Inner Product Argument over Augmented KZG Commitments

To reduce the linear proof overhead in BGW-style ADMPC, we first introduce $\mathsf{IPA}_{\mathsf{KZG}}$, a specialized inner-product argument protocol built upon our augmented KZG commitment scheme. As illustrated in Figure 5, we adapt the inner-product argument proposed in Bulletproofs [14] as a building block to enable a prover to convince a verifier that, for every multiplication gate $k \in [M]$, the evaluations of the polynomials committed to by $\{C_{a_k}, C_{b_k}, C_{c_k}\}$ satisfy a multiplicative relation. Specifically, $\mathsf{IPA}_{\mathsf{KZG}}$ proves that each evaluated secret share $f_{c_k}(i_c)$ is exactly the product of $f_{a_k}(i_a)$ and $f_{b_k}(i_b)$, while simultaneously certifying that all $3M$ evaluations are consistent with their respective KZG commitments through aggregated verification. In particular, $\mathsf{IPA}_{\mathsf{KZG}}$ attests to the following statement:

$$\mathsf{IPA}_{\mathsf{KZG}} \begin{bmatrix} \{(f_{x_k}(i_x), \hat{f}_{x_k}(i_x), w_{x_k})\}_{x\in\{a,b,c\},k\in[M]} : \\ f_{a_k}(i_a) \cdot f_{b_k}(i_b) = f_{c_k}(i_c) \wedge \\ \mathsf{AggPubVerEval}(SP, i_x, g^{f_x(i_x)} h^{\hat{f}_x(i_x)}, W_x, \\ \{C_{x_k}\}_{k\in[M]}) = 1, \quad \forall x \in \{a,b,c\}, k \in [M] \end{bmatrix}$$

Concretely, the prover first aggregates the evaluation proofs for each polynomial using the Fiat–Shamir challenge γ via our augmented KZG interfaces. Using evaluation commitments $g^{f_{x_k}(i_x)} h^{\hat{f}_{x_k}(i_x)}$ as Pedersen-style commitments to polynomial evaluations at the designated points, the prover invokes the standard inner-product argument to prove the multiplicative relations. This construction is suitable for our per-epoch evaluation phase, where all M multiplication gates within a layer are mutually independent. This independence allows us to certify all M gates via a single inner-product argument, without introducing any additional circuit-level constraints. The verifier recomputes γ, checks each aggregated evaluation proof via AggPubVerEval, and finally verifies the inner product proof to ensure that all claimed products are correct. This design yields an inner-product proof of size $O(\log M)$, while ensuring that the claimed multiplicative relations are consistent with the committed polynomial evaluations.

Security Properties. Our protocol inherits the security guarantees of the inner product argument from Bulletproofs [14]. In particular, it enjoys *perfect completeness*, *statistical zero-knowledge*, and *computational soundness under the discrete logarithm assumption*, as established in Theorem 5 of [14].

Inner Product Argument over KZG Commitments $\mathsf{IPA}_{\mathsf{KZG}}$

The prover $\mathcal{P}$ and the verifier $\mathcal{V}$ both know the public statement $(SP, \{C_{a_k}, C_{b_k}, C_{c_k}\}_{k\in[M]}, i_a, i_b, i_c, \mathcal{P}_{id})$ where $\{C_{x_k}\}_{x\in\{a,b,c\},k\in[M]}$ are KZG polynomial commitments and $\{i_x\}_{x\in\{a,b,c\}}$ are points to evaluate polynomials. $\mathcal{P}$ has private witness $\{(f_{x_k}(i_x), \hat{f}_{x_k}(i_x), w_{x_k})\}_{x\in\{a,b,c\},k\in[M]}$, s.t. $\forall x \in \{a,b,c\}$ and $k \in [M]$, $\mathsf{VerifyEval}(SP, C_{x_k}, i_x, f_{x_k}(i_x), \hat{f}_{x_k}(i_x), w_{x_k}) = 1$.

// **Prover** $\mathcal{P}$

$\mathsf{IP}_{\mathsf{KZG}}.\mathcal{P}(\{C_{x_k}, i_x, f_{x_k}(i_x), \hat{f}_{x_k}(i_x), w_{x_k}\}_{x\in\{a,b,c\},k\in[M]})$:

(1) compute challenge $\gamma = \mathcal{H}(SP, \mathcal{P}_{id}, \{C_{c_k}\}_{k\in[M]})$ and aggregate $W_x = \prod_{k\in[M]} (w_{x_k})^{\gamma^{k-1}}$ for each $x \in \{a,b,c\}$;

(2) invoke the inner product argument prover on $\{(f_{x_k}(i_x), \hat{f}_{x_k}(i_x))\}_{x\in\{a,b,c\},k\in[M]}$ as witness, and obtain the proof π regarding the product relation of $\{(g^{f_{a_k}(i_a)} h^{\hat{f}_{a_k}(i_a)}, g^{f_{b_k}(i_b)} h^{\hat{f}_{b_k}(i_b)}, g^{f_{c_k}(i_c)} h^{\hat{f}_{c_k}(i_c)})\}_{k\in[M]}$;

(3) output proof $= (\{(g^{f_{x_k}(i_x)} h^{\hat{f}_{x_k}(i_x)}, W_x)\}_{x\in\{a,b,c\},k\in[M]}, \pi)$.

// **Verifier** $\mathcal{V}$

$\mathsf{IP}_{\mathsf{KZG}}.\mathcal{V}(\{(C_{x_k}, i_k)\}_{x\in\{a,b,c\},k\in[M]}, \text{proof})$:

(1) parse proof as $(\{(g^{f_{x_k}(i_x)} h^{\hat{f}_{x_k}(i_x)}, W_x)\}_{x\in\{a,b,c\},k\in[M]}, \pi)$;

(2) recompute the challenge γ to aggregate $\{g^{f_{x_k}(i_x)} h^{\hat{f}_{x_k}(i_x)}\}_{k\in[M]}$ as $g^{f_x(i_x)} h^{\hat{f}_x(i_x)}$ for each $x \in \{a,b,c\}$;

(3) verify $\mathsf{AggPubVerEval}(SP, i_x, g^{f_x(i_x)} h^{\hat{f}_x(i_x)}, W_x, \{C_{x_k}\}_{k\in[M]}) = 1$ for each $x \in \{a,b,c\}$;

(4) verify the inner product proof π regarding $\{(g^{f_{a_k}(i_a)} h^{\hat{f}_{a_k}(i_a)}, g^{f_{b_k}(i_b)} h^{\hat{f}_{b_k}(i_b)}, g^{f_{c_k}(i_c)} h^{\hat{f}_{c_k}(i_c)})\}_{k\in[M]}$, return 1 if all checks pass and 0 otherwise.

Figure 5: Inner product relation for evaluations of KZG-committed polynomials.

6.3 Construction of the BatchMul protocol

Building upon our $\mathsf{IPA}_{\mathsf{KZG}}$ construction, we now present our batch multiplication evaluation protocol, denoted as BatchMul. BatchMul is designed as a specialized variant of AggTrans that resolves interaction bottlenecks by deferring the verifiable degree-reduction procedure to the hand-off phase. The protocol ensures multiplication correctness, ensuring that the outputs of all multiplication gates are correctly computed and faithfully encoded as fresh degree-t sharings for the next committee. Algorithm 2 summarizes the procedure, which proceeds in the following main phases:

(1) *Proof generation and resharing (lines 101–106).* Each server $P_i^l \in \mathcal{P}^l$ starts with degree-$2t$ product shares $\langle c_k \rangle_i^l$ with randomness $\langle \hat{c}_k \rangle_i^l$ provided by the non-interactive computation phase. To attest to the correctness of these multiplications, P_i^l invokes $\mathsf{IPA}_{\mathsf{KZG}}$ to produce a single logarithmic-size proof π_i that simultaneously verifies $\langle c_k \rangle_i^l = [a_k]_i^l \cdot [b_k]_i^l$ for all $k \in [M]$ and ensures all evaluations are consistent with their respective committed polynomials. Following this, all products $\{\langle c_k \rangle_i^l\}_{k\in[M]}$ are reshared using a BACSS instance. During this process, P_i^l broadcasts the reshared products along with their new KZG commitments and includes both the evaluation proofs and the inner product proof π_i within proof_i.

(2) *Verification via* $\mathsf{IPA}_{\mathsf{KZG}}$ *(lines 201–205).* Upon receiving the resharing and their associated proofs from P_i^l, each server $P_j^{l+1} \in \mathcal{P}^{l+1}$ invokes our $\mathsf{IPA}_{\mathsf{KZG}}.\mathcal{V}$ to validate all multiplication gates in one shot. This verification step ensures that every reshared product $\langle c_k \rangle_i^l$ is indeed the correct multiplication of the corresponding input shares $[a_k]_i^l$ and $[b_k]_i^l$, while also guaranteeing that all evaluations are consistent with their committed polynomials.
As in the aggregated hand-off protocol, the same adversarial attack remains possible in this setting. A corrupted dealer may submit shares $[a']$, $[b']$, and a multiplication result

Algorithm 2 Batch Multiplication for $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$

Public Inputs: KZG commitments $\{C^l_{a_k}, C^l_{b_k}\}_{k\in[M]}$ and other system parameters

Private Inputs: P^l_i holds degree-t shares $\{[a_k]^l_i, [b_k]^l_i\}_{k\in[M]}$ with randomness $\{[\hat{a}_k]^l_i, [\hat{b}_k]^l_i\}_{k\in[M]}$, evaluation proofs $\{w^l_{a_k i}, w^l_{b_k i}\}_{k\in[M]}$, and degree-$2t$ product shares $\{\langle c_k\rangle^l_i\}_{k\in[M]}$ with randomness $\{\langle \hat{c}_k\rangle^l_i\}_{k\in[M]}$

Public Outputs: New KZG commitments $\{C^{l+1}_{c_k}\}_{k\in[M]}$

Private Outputs: P^{l+1}_j obtains $\{[c_k]^{l+1}_j\}_{k\in[M]}$ with randomness $\{[\hat{c}_k]^{l+1}_j\}_{k\in[M]}$ and evaluation proofs $\{w^{l+1}_{c_k i}\}_{k\in[M]}$

Server P^l_i (in committee $\mathcal{P}^l$):

// 1. Evaluate multiplications via $\mathsf{IPA}_{\mathsf{KZG}}$ and reshare

101: **for** $k \in [M]$

102: $(C^{l+1}_{c_k i}, \hat{f}^{l+1}_{c_k i}(\cdot)) \leftarrow \mathsf{PolyCom}(SP, f^{l+1}_{c_k i}(\cdot))$

// s.t. $f^{l+1}_{c_k i}(0) = \langle c_k\rangle^l_i$, $\hat{f}^{l+1}_{c_k i}(0) = \langle \hat{c}_k\rangle^l_i$

103: **for** $j \in [0, n]$

104: $w^{l+1}_{c_k i,j} \leftarrow \mathsf{ProveEval}(SP, j, f^{l+1}_{c_k i}(\cdot), \hat{f}^{l+1}_{c_k i}(\cdot))$

105: $\mathsf{proof}_i \leftarrow \mathsf{IP}_{\mathsf{KZG}}.\mathcal{P}\left(\left\{\begin{array}{l}(C^l_{a_k}, i, [a_k]^l_i, [\hat{a}_k]^l_i, w^l_{a_k,i}),\\ (C^l_{b_k}, i, [b_k]^l_i, [\hat{b}_k]^l_i, w^l_{b_k,i}),\\ (C^{l+1}_{c_k i}, 0, \langle c_k\rangle^l_i, \langle \hat{c}_k\rangle^l_i, w^{l+1}_{c_k i,0})\end{array}\right\}\right)$ for $k \in [M]$

106: invoke $\mathsf{BACSS_share}_i$ as dealer to reshare $\{\langle c_k\rangle^l_i\}_{k\in[M]}$ using KZG commitments $\{C^{l+1}_{c_k i}\}_{k\in[M]}$ with broadcasting $(\{(C^l_{a_k}, C^l_{b_k})\}_{k\in[M]}, \mathsf{proof}_i)$

Server P^{l+1}_j (in committee $\mathcal{P}^{l+1}$):

// 2. Verify multiplication correctness via $\mathsf{IPA}_{\mathsf{KZG}}$

201: $T_j \leftarrow \emptyset$, $\mathtt{mul_shares}_j \leftarrow \emptyset$

202: activate $\mathsf{BACSS_share}_i$ instance for every $i \in [n]$

203: **upon** receiving valid shares $\{([\langle c_k\rangle^l_i]^{l+1}_j, [\langle \hat{c}_k\rangle^l_i]^{l+1}_j, C^{l+1}_{c_k i}, w^{l+1}_{c_k i,j})\}_{k\in[M]}$ and $(\{(C^l_{a_k}, C^l_{b_k})\}_{k\in[M]}, \mathsf{proof}_i)$ from $\mathsf{BACSS_share}_i$

// Validity verified via VerifyEval

// s.t. $[\langle c_k\rangle^l_i]^{l+1}_j = f^{l+1}_{c_k i}(j)$, $[\langle \hat{c}_k\rangle^l_i]^{l+1}_j = \hat{f}^{l+1}_{c_k i}(j)$

204: **if** $\mathsf{IP}_{\mathsf{KZG}}.\mathcal{V}\left(\left\{\begin{array}{l}(C^l_{a_k}, i),\\ (C^l_{b_k}, i),\\ (C^{l+1}_{c_k i}, 0)\end{array}\right\}_{k\in[M]}, \mathsf{proof}_i\right) = 1$

205: $T_j \leftarrow T_j \cup \{i\}$

// 3. Agree on subset with matching commitments

206: **if** $|T_j| \geq 2t+1$ **and** $\exists T'_j \subseteq T_j$ with $|T'_j| = 2t+1$ such that the polynomial commitments $C^l_{a_k}$ and $C^l_{b_k}$ in T'_j match for all $k \in [M]$

207: call MVBA using T'_j as input and T_j as state

// MVBA's predicate checks if $T_j \supseteq$ input

208: **wait** for MVBA output T, w.l.o.g, let $T = \{P^l_1, \cdots, P^l_{2t+1}\}$

// 4. Interpolate new secret shares

209: **for** $k \in [M]$

210: $[c_k]^{l+1}_j \leftarrow \Sigma_{i=1}^{2t+1} \lambda_i [\langle c_k\rangle^l_i]^{l+1}_j$

// λ_i is the Lagrange coeff. for eval at 0

// update the corresponding KZG commitments and randomness for the new shares

211: $\mathtt{mul_shares} \leftarrow \mathtt{mul_shares} \cup \{[c_k]^{l+1}_j\}_{k\in[B]}$

212: **return** $\mathtt{mul_shares}$

$\langle c'\rangle$ that satisfy the local multiplication relation, yet are inconsistent with the secrets committed for epoch C^l. To defend such attacks, the protocol proceeds to an agreement phase based on matching commitments.

(3) *Agreement on subset with matching commitments (lines 206–208).* Similar to AggTrans, this phase enforces global consistency by anchoring dealer sharings to be verifiably consistent with the state committed for the previous epoch. After the local verification phase, each server P^{l+1}_j maintains a candidate set T_j of dealers whose multiplication shares and proofs have passed validation. To achieve global consistency across the committee, P^{l+1}_j identifies a subset $T'_j \subseteq T_j$ of size $2t+1$ such that all dealers in T'_j present identical polynomial commitments $\{C^l_{a_k}, C^l_{b_k}\}_{k\in[M]}$ for their respective input shares. Once such a subset is found, P^{l+1}_j engages in an MVBA instance using T'_j as input to reach agreement on a common set of valid dealers. This consensus step ensures that only multiplication shares bound to the same set of commitments are accepted, thereby preventing the adversary from injecting inputs that are inconsistent or unrelated to the intended computation.

(4) *Interpolation of new multiplication sharings (lines 209–212).* After consensus is reached on a valid dealer set $T = \{P^l_1, \ldots, P^l_{2t+1}\}$ via MVBA, each server P^{l+1}_j reconstructs its share of every multiplication result $[c_k]^{l+1}_j$ by performing Lagrange interpolation over the shares $\{[\langle c_k\rangle^l_i]^{l+1}_j\}_{i\in T}$ for all $k \in [M]$. This yields fresh degree-t sharings of the multiplication outputs across $\mathcal{P}^{l+1}$, ensuring that the results remain faithful to those computed in epoch C^l and thereby upholding the multiplication correctness.

Security of BatchMul**.** The security properties of BatchMul, including *termination*, *transfer consistency*, *multiplication correctness*, and *secrecy*, are formally stated and proven in Appendix C.

Complexity of BatchMul**.** To reshare M multiplication outputs, $\Theta(n)$ BACSS instances are executed in parallel, each contributing $O(\kappa\, n \cdot M)$ communication, for a total of $O(\kappa\, n^2 \cdot M)$. Each dealer additionally sends an inner-product proof of size $O(\kappa \log M)$, adding $O(\kappa\, n \log M)$ overall. Finally, a single MVBA agreement incurs

$O(\kappa\, n^3)$ communication. Thus, the total complexity of BatchMul is $O(\kappa\, n^2 \cdot M + \kappa\, n^3)$.

7 Implementation and Evaluation

In this section, we implement a prototype of Continuum and evaluate its performance through system setup, sub-protocol benchmarks, and comparisons with AD-MPC and Dumbo-MPC.

7.1 Implementation and Evaluation Setup

Implementation. We have implemented a prototype of Continuum. The core protocol components, including AggTrans and BatchMul, are implemented in Python 3 as forks of the open-source implementations of AD-MPC [46] and Dumbo-MPC [44]. These p2p channels use unauthenticated TCP sockets, and concurrent tasks are managed via Python's `asyncio`[3] library. Continuum relies on pairing-friendly elliptic curve BLS12-381, instantiating the KZG polynomial commitment scheme [38] with aggregation interfaces and the inner product argument [14] over augmented commitments. All finite field operations are performed over a 256-bit prime field $\mathbb{Z}_q$ for Shamir secret sharing.

Evaluation Setup. Each committee comprises $n = 3t+1$ servers, tolerating up to t corruptions. The client set includes $k = n$ clients, of which up to $c = t$ may also be corrupted. In our deployment, we omit the committee election and directly configure committees to be disjoint across epochs, consistent with our asynchronous dynamic model described in Section 3.1. Specifically, servers from different machines but aligned by the same port identifier are grouped into the same committee. To evaluate the efficiency and robustness of our design, we focus on four central questions that align with our core contributions: (i) we assess how efficient AggTrans is compared to the original AD-MPC design; (ii) we examine whether BatchMul improves performance by eliminating preprocessing and compressing correctness proofs; (iii) we investigate how effectively Continuum achieves guaranteed output delivery in asynchronous dynamic settings compared to AD-MPC and Dumbo-MPC; and (iv) we study how circuit size influences the latency of Continuum relative to the two baselines.

For the evaluation of AggTrans and BatchMul, we use rectangular arithmetic circuits with depth d and width w. For transfer evaluation, the circuit consists solely of linear gates, while for multiplication evaluation, the circuit consists solely of multiplication gates. Client inputs are fixed at $2w$, and gate inputs are structured sequentially. To compare Continuum with AD-MPC and Dumbo-MPC, we use rectangular arithmetic circuits with depth d and width w, maintaining a 1:1 ratio of multiplication to linear gates per layer. We then fix the circuit width and test the protocol latency by varying the circuit depth.

We deploy our experiments on 16 geographically distributed WAN-based instances (type `C6.4XLARGE32`, 16-core CPU, 32GB RAM, 50GB SSD, 500Mbps bandwidth) running Debian 12.0. Each experiment is repeated twenty times, and we report the average results.

7.2 Evaluation Results

Transfer Protocol Evaluation. We set up a layered arithmetic circuit with depth $d = 6$ and width $w = 100$, consisting solely of linear gates, and evaluate the latency of the transfer protocols under committee sizes $n = 4, 8, 12$, and 16. As shown in Figure 6, the latency of AD-MPC is dominated by two components: the generation of masking randomness (RandGen) and the transfer protocol (ADtrans). In contrast, our AggTrans protocol eliminates the RandGen phase entirely, whose cost grows from 4.39s at $n = 4$ to 8.83s at $n = 16$. Moreover, even when focusing only on the transfer stage, AggTrans consistently outperforms ADtrans by a factor of 2.8–4.3×. This improvement arises from the optimized implementation of KZG commitments, the heavier computational overhead of hbPolyCom [47] in AD-MPC, and the lower communication complexity of AggTrans. Overall, AggTrans achieves a 4.5–5.5× reduction in transfer latency compared to the mask-and-reconstruct strategy of AD-MPC.

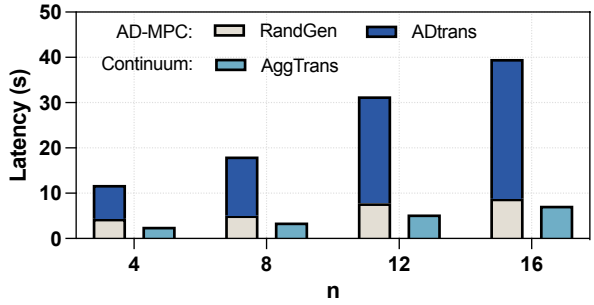


Figure 6: Latency of transfer protocols under all-linear circuits. We fix the circuit depth $d = 6$ and width $w = 100$, and circuits consist solely of linear gates. We vary the committee size $n \in \{4, 8, 12, 16\}$ and report the total transfer latency. For AD-MPC, the bar is decomposed into RandGen and ADtrans; in contrast, our Continuum requires only AggTrans.

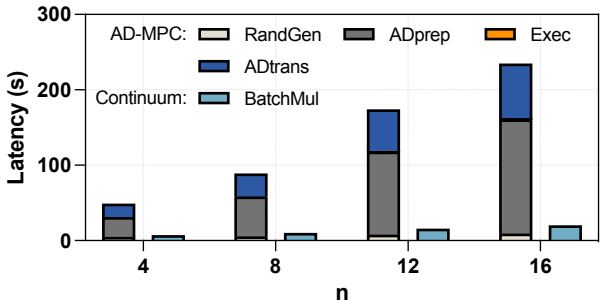


Figure 7: Latency of multiplication protocols under all-multiplication circuits. The circuit depth is fixed at $d = 6$ and width at $w = 100$, with all gates being multiplication gates. We vary the committee size $n \in \{4, 8, 12, 16\}$ and report the total latency. For AD-MPC, the bar is decomposed into ADprep, RandGen, Exec, and ADtrans, while our Continuum requires only BatchMul.

Multiplication Protocol Evaluation. We adopt the same circuit setup as in the transfer evaluation (rectangular layered circuits

[3] https://github.com/python/cpython

Table 2: Proof size for multiplication proofs with varying numbers of gates.

Protocols	Multiplication gates M =		
	2	20	200
Dumbo-MPC [44]	2366 B	23648 B	236426 B
Continuum	2614 B	4238 B	5456 B

with fixed depth and width), except that all gates are multiplication gates, and we vary the committee size as before. As shown in Figure 7, the latency of AD-MPC consists of four components: ADprep for generating Beaver triples, RandGen for distributing masking randomness required by the ADtrans protocol, Exec for consuming triples to evaluate multiplication gates, and finally ADtrans to transfer secret-shared outputs to the next committee. In contrast, our BatchMul protocol follows the BGW paradigm: it requires no preprocessing, the computation phase is purely local without communication overhead, and the only communication occurs in the hand-off phase when multiplication outputs are reshared to the next committee. As a result, BatchMul achieves up to an 8.0× reduction in overall latency. We further evaluate the size of zero-knowledge proofs and compare with Dumbo-MPC, as shown in Table 2. The results demonstrate that as the number of multiplication gates increases, the proof size in Continuum grows much more slowly and remains asymptotically smaller than that of Dumbo-MPC.

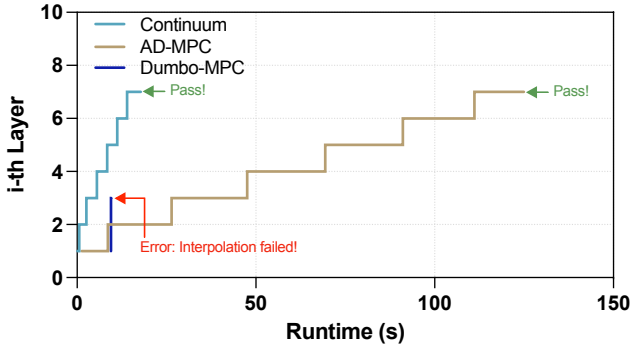


Figure 8: Progress of Continuum**, AD-MPC, and Dumbo-MPC on a circuit of width $w = 100$ and depth $d = 6$ with an equal number of multiplication and linear gates per epoch, under committee size $n = 16$. The 7th layer corresponds to the output stage where clients receive and reconstruct their shares.**

The impact of asynchronous dynamic setting on guaranteed output delivery. We set the circuit width $w = 100$, depth $d = 6$, and committee size $n = 16$, tolerating up to $t = 5$ corrupted parties. Each epoch contains an equal number of multiplication and linear gates, and the 7-th epoch represents the output stage where clients reconstruct and verify the final shares. We then evaluate the progress of Continuum, AD-MPC, and Dumbo-MPC under this configuration. The results are shown in Figure 8.

For Dumbo-MPC, we simulate an honest server going offline during the 4-th epoch. Because Dumbo-MPC assumes a static committee, the remaining honest servers fail to receive the required number of shares during the multiplication evaluation. Consequently, the protocol fails to interpolate correct polynomials, leading to termination (Error: Interpolation failed!). This illustrates that dynamic conditions can compromise protocols that otherwise guarantee output delivery. In contrast, both AD-MPC and Continuum successfully achieve GOD under the same asynchronous dynamic setting. Moreover, AD-MPC incurs 107.15s higher end-to-end latency than Continuum, highlighting the concrete efficiency improvements of our enhanced design.

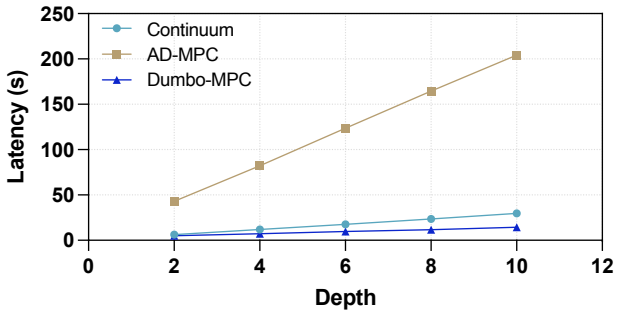


Figure 9: Latency comparison of Continuum**, AD-MPC, and Dumbo-MPC with circuit width $w = 100$, committee size $n = 16$, and varying circuit depth $d \in \{2, 4, 6, 8, 10\}$.**

The impact of circuit size on latency. We evaluate the impact of circuit size on the latency of the three protocols. We fix the circuit width $w = 100$ and the committee size $n = 16$, while varying the circuit depth $d \in \{2, 4, 6, 8, 10\}$. As shown in Figure 9, AD-MPC exhibits substantially higher latency than both Continuum and Dumbo-MPC across all depths. When the circuit depth is small, Continuum even outperforms Dumbo-MPC, achieving lower latency. However, as the depth increases, the overhead introduced by dynamic settings becomes more pronounced, which accounts for the steeper slope of Continuum compared to Dumbo-MPC.

8 Conclusion

We presented Continuum, a concretely efficient asynchronous dynamic MPC protocol with guaranteed output delivery. By introducing AggTrans to eliminate per-epoch masking randomness and BatchMul for efficient multiplication gate evaluation, we address the concrete inefficiencies of prior designs. Our implementation demonstrates both strong security guarantees and substantial performance improvements in asynchronous dynamic environments.

Ethical Considerations

We ensure that our research adheres to the four ethical principles outlined in the Menlo Report: "Respect for Persons" "Beneficence," "Justice," and "Respect for Law and Public Interest." In addition, following the USENIX Security guidelines, we explicitly identify stakeholders, analyze potential risks, and explain our decision to pursue and publish this research. The specific considerations are as follows:

- **Stakeholders:** The primary stakeholders of this research include (i) the academic and industrial communities that may benefit from more efficient and secure MPC protocols, (ii) potential users of MPC-as-a-Service infrastructures that rely on scalable and robust protocols, and (iii) the broader public who may be indirectly affected by the deployment of privacy-preserving technologies. No direct human subjects were involved in the research process.
- **Respect for Persons:** Our study does not involve experiments on human participants, personal data collection, or interaction with live production systems. All evaluations were conducted on controlled, open-source implementations. All external code bases and libraries used in experiments are properly cited, ensuring respect for intellectual contributions.
- **Beneficence:** The protocol design and experiments do not process personal or sensitive data, thus eliminating risks of privacy violations. The research offers substantial benefits by reducing communication complexity and improving scalability of asynchronous DMPC, thereby strengthening the feasibility of real-world MPC deployments. To mitigate potential risks of misuse, we release our code with clear licensing terms to prevent malicious use.
- **Justice:** The benefits of our work are broadly distributed: the proposed protocols can be applied to diverse application domains (e.g., privacy-preserving machine learning, secure voting, and confidential smart contracts). No specific community bears disproportionate risks or burdens as a result of our research, ensuring fairness in the distribution of both costs and benefits.
- **Respect for Law and Public Interest:** All experiments were conducted in compliance with relevant laws and regulations. We open-source our implementation and provide detailed descriptions of the protocols to enable transparency and accountability. At the same time, we refrain from including sensitive implementation details that could facilitate adversarial exploitation.
- **Decision:** After evaluating the ethical principles and potential stakeholder impacts, we conclude that conducting and publishing this research is ethically justified. The work strengthens privacy-enhancing technologies, poses negligible risks to individuals or organizations, and contributes positively to the security and public interest of distributed computing systems.

Open Science Policy

To support the reproducibility of our experimental results, we provide an anonymized artifact repository at: https://anonymous.4open.science/r/ad-mpc-pp-C456. This repository contains the full implementation of Continuum, including all source code, evaluation scripts, and configuration files. We provide a comprehensive `README` with step-by-step instructions for environment setup and the execution of benchmarks used to obtain the results presented in this paper. These artifacts include everything necessary to evaluate the core contributions of this paper. We commit to releasing the source code under an open-source license and archiving it with a permanent DOI upon publication.

Generative AI Usage

We used ChatGPT to assist with drafting and refining parts of the manuscript and to produce an initial code skeleton during early-stage implementation. All protocol designs, algorithms, security arguments, and experimental logic were developed by the authors. The AI-generated code drafts were manually reviewed, completed, and tested by the authors, and all experimental results were reproduced from the final, human-verified implementation.

References

[1] Ittai Abraham, Gilad Asharov, Arpita Patra, and Gilad Stern. 2023. Perfectly Secure Asynchronous Agreement on a Core Set in Constant Expected Time. *IACR Cryptol. ePrint Arch.* 2023 (2023), 1130.

[2] Anasuya Acharya, Carmit Hazay, Vladimir Kolesnikov, and Manoj Prabhakaran. 2022. Scales: Mpc with small clients and larger ephemeral servers. In *Theory of Cryptography Conference*. Springer, 502–531.

[3] Akhil Bandarupalli, Xiaoyu Ji, Aniket Kate, Chen-Da Liu-Zhang, and Yifan Song. 2025. Computationally efficient asynchronous MPC with linear communication and low additive overhead. In *Annual International Cryptology Conference*. Springer, 261–294.

[4] Aritra Banerjee, Michael Clear, and Hitesh Tewari. 2021. zkhawk: Practical private smart contracts from mpc-based hawk. In *2021 3rd Conference on Blockchain Research & Applications for Innovative Networks and Services (BRAINS)*. IEEE, 245–248.

[5] Assi Barak, Martin Hirt, Lior Koskas, and Yehuda Lindell. 2018. An end-to-end system for large scale P2P MPC-as-a-service and low-bandwidth MPC for weak participants. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 695–712.

[6] Carsten Baum, James Hsin-yu Chiang, Bernardo David, and Tore Kasper Frederiksen. 2023. Eagle: Efficient privacy preserving smart contracts. In *International Conference on Financial Cryptography and Data Security*. Springer, 270–288.

[7] Michael Ben-Or, Ran Canetti, and Oded Goldreich. 1993. Asynchronous secure computation. In *Proceedings of the twenty-fifth annual ACM symposium on Theory of computing*. 52–61.

[8] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. 2019. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*. 351–371.

[9] Rishabh Bhadauria, James Hsin-yu Chiang, Divya Ravi, Jure Sternad, and Sophia Yakoubov. 2024. Minimizing the Use of the Honest Majority in YOSO MPC with Guaranteed Output Delivery. *Cryptology ePrint Archive* (2024).

[10] Alexander Bienstock, Daniel Escudero, and Antigoni Polychroniadou. 2023. On Linear Communication Complexity for (Maximally) Fluid MPC. *Cryptology ePrint Archive* (2023).

[11] Alexander Bienstock, Daniel Escudero, and Antigoni Polychroniadou. 2024. Perfectly Secure Fluid MPC with Abort and Linear Communication Complexity. *Cryptology ePrint Archive* (2024).

[12] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. 2020. Efficient polynomial commitment schemes for multiple points and polynomials. *Cryptology ePrint Archive* (2020).

[13] Lennart Braun, Ivan Damgård, and Claudio Orlandi. 2023. Secure multiparty computation from threshold encryption based on class groups. In *Annual International Cryptology Conference*. Springer, 613–645.

[14] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. 2018. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*. IEEE, 315–334.

[15] Christian Cachin, Klaus Kursawe, Frank Petzold, and Victor Shoup. 2001. Secure and efficient asynchronous broadcast protocols. In *Annual International Cryptology Conference*. Springer, 524–541.

[16] Ran Canetti, Sebastian Kolby, Divya Ravi, Eduardo Soria-Vazquez, and Sophia Yakoubov. 2023. Taming Adaptivity in YOSO Protocols: The Modular Way. In *Theory of Cryptography Conference*. Springer, 33–62.

[17] Ignacio Cascudo, Bernardo David, Lydia Garms, and Anders Konring. 2022. YOLO YOSO: fast and simple encryption and secret sharing in the YOSO model. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 651–680.

[18] Arka Rai Choudhuri, Aarushi Goel, Matthew Green, Abhishek Jain, and Gabriel Kaptchuk. 2021. Fluid MPC: secure multiparty computation with dynamic participants. In *Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part II 41*. Springer, 94–123.

[19] Ashish Choudhury, Martin Hirt, and Arpita Patra. 2013. Asynchronous multiparty computation with linear communication complexity. In *International Symposium on Distributed Computing*. Springer, 388–402.

[20] Ashish Choudhury and Arpita Patra. 2016. An efficient framework for unconditionally secure multiparty computation. *IEEE Transactions on Information Theory* 63, 1 (2016), 428–468.

[21] Ashish Choudhury and Arpita Patra. 2023. On the Communication Efficiency of Statistically Secure Asynchronous MPC with Optimal Resilience. *Journal of Cryptology* 36, 2 (2023), 13.

[22] Ivan Damgård, Daniel Escudero, and Antigoni Polychroniadou. 2021. Phoenix: Secure computation in an unstable network with dropouts and comebacks. *Cryptology ePrint Archive* (2021).

[23] Ivan Bjerre Damgård, Simon Holmgaard Kamp, Julian Loss, and Jesper Buus Nielsen. 2025. Asynchronous YOSO a la Paillier. *Cryptology ePrint Archive* (2025).

[24] Sourav Das, Zhuolun Xiang, Lefteris Kokoris-Kogias, and Ling Ren. 2023. Practical asynchronous high-threshold distributed key generation and distributed polynomial sampling. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5359–5376.

[25] Sourav Das, Zhuolun Xiang, and Ling Ren. 2021. Asynchronous data dissemination and its applications. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 2705–2721.

[26] Sourav Das, Zhuolun Xiang, and Ling Ren. 2024. Powers of Tau in Asynchrony. In *Proceedings of the 2024 Network and Distributed System Security Symposium*.

[27] Sourav Das, Thomas Yurek, Zhuolun Xiang, Andrew Miller, Lefteris Kokoris-Kogias, and Ling Ren. 2022. Practical asynchronous distributed key generation. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2518–2534.

[28] Bernardo David, Giovanni Deligios, Aarushi Goel, Yuval Ishai, Anders Konring, Eyal Kushilevitz, Chen-Da Liu-Zhang, and Varun Narayanan. 2023. Perfect MPC over layered graphs. In *Annual International Cryptology Conference*. Springer, 360–392.

[29] Giovanni Deligios, Anders Konring, Chen-Da Liu-Zhang, and Varun Narayanan. 2025. Statistical Layered MPC. In *Theory of Cryptography Conference*. Springer, 362–394.

[30] Andreas Erwig, Sebastian Faust, and Siavash Riahi. 2021. Large-scale non-interactive threshold cryptosystems in the yoso model. *Cryptology ePrint Archive* (2021).

[31] Amos Fiat and Adi Shamir. 1986. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*. Springer, 186–194.

[32] Craig Gentry, Shai Halevi, Hugo Krawczyk, Bernardo Magri, Jesper Buus Nielsen, Tal Rabin, and Sophia Yakoubov. 2021. Yoso: You only speak once: Secure mpc with stateless ephemeral roles. In *Annual International Cryptology Conference*. Springer, 64–93.

[33] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. 2017. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th symposium on operating systems principles*. 51–68.

[34] Oded Goldreich, Silvio Micali, and Avi Wigderson. 2019. How to play any mental game, or a completeness theorem for protocols with honest majority. In *Providing Sound Foundations for Cryptography: On the Work of Shafi Goldwasser and Silvio Micali*. 307–328.

[35] Vipul Goyal, Chen-Da Liu-Zhang, and Yifan Song. 2024. Towards Achieving Asynchronous MPC with Linear Communication and Optimal Resilience. *Cryptology ePrint Archive* (2024).

[36] Jens Groth and Victor Shoup. 2022. Design and analysis of a distributed ECDSA signing service. *Cryptology ePrint Archive* (2022).

[37] Bingyong Guo, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. 2020. Dumbo: Faster asynchronous bft protocols. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 803–818.

[38] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. 2010. Constant-size commitments to polynomials and their applications. In *International conference on the theory and application of cryptology and information security*. Springer, 177–194.

[39] Sebastian Kolby, Divya Ravi, and Sophia Yakoubov. 2022. Constant-round YOSO MPC without setup. *Cryptology ePrint Archive* (2022).

[40] Ziyao Liu, Ivan Tjuawinata, Chaoping Xing, and Kwok-Yan Lam. 2020. MPC-enabled privacy-preserving neural network training against malicious attack. *arXiv preprint arXiv:2007.12557* (2020).

[41] Chen-Da Liu-Zhang, Elisaweta Masserova, João Ribeiro, Pratik Soni, and Sri AravindaKrishnan Thyagarajan. 2024. Improved YOSO Randomness Generation with Worst-Case Corruptions. *Cryptology ePrint Archive* (2024).

[42] Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew Miller. 2019. Honeybadgermpc and asynchromix: Practical asynchronous mpc and its application to anonymous communication. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 887–903.

[43] Rahul Rachuri and Peter Scholl. 2022. Le mans: Dynamic and fluid MPC for dishonest majority. In *Annual International Cryptology Conference*. Springer, 719–749.

[44] Yuan Su, Yuan Lu, Jiliang Li, Yuyi Wang, Chengyi Dong, and Qiang Tang. 2025. Dumbo-mpc: efficient fully asynchronous mpc with optimal resilience. In *USENIX Security*, Vol. 2024.

[45] Andrew Chi-Chih Yao. 1986. How to generate and exchange secrets. In *27th annual symposium on foundations of computer science (Sfcs 1986)*. IEEE, 162–167.

[46] Wenxuan Yu, Minghui Xu, Bing Wu, Sisi Duan, and Xiuzhen Cheng. 2025. AD-MPC: Asynchronous Dynamic MPC with Guaranteed Output Delivery. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. 2892–2906.

[47] Thomas Yurek, Licheng Luo, Jaiden Fairoze, Aniket Kate, and Andrew Miller. 2022. hbACSS: How to Robustly Share Many Secrets. In *Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 24–28.

A Security Proofs for KZG Polynomial Commitment with Aggregation Interfaces

In this section, we formally prove that our proposed KZG scheme with aggregation interfaces satisfies the properties of *correctness*, *public verifiability*, *evaluation binding*, *polynomial binding*, *aggregation binding*, and *hiding* under the hardness assumptions of the discrete logarithm and t-strong Diffie–Hellman problems. We begin by recalling these properties formally, and then proceed to provide rigorous proofs.

<u>*Correctness:*</u> Given any $SP \leftarrow \mathsf{Setup}(1^\kappa, t)$ and any collection of polynomials $\{f_j(\cdot)\}_{j\in B} \subseteq \mathbb{Z}_q[x]$, let the corresponding commitments $(C_j, \hat{f}_j(\cdot)) \leftarrow \mathsf{PolyCom}(SP, f_j(\cdot))$ be honestly generated. Then:

- For each $j \in B$, any output of $\mathsf{ProveEval}(SP, i, f_j(\cdot), \hat{f}_j(\cdot))$ always passes the verification of $\mathsf{VerifyEval}$.
- Any output of $\mathsf{AggPubProEval}(SP, i, \{f_j(\cdot), \hat{f}_j(\cdot), C_j\}_{j\in B})$ always passes the verification of $\mathsf{AggPubVerEval}$.

<u>*Public verifiability:*</u> Consider any $SP \leftarrow \mathsf{Setup}(1^\kappa, t)$ and a set of polynomials $\{f_j(\cdot)\}_{j\in B} \subseteq \mathbb{Z}_q[x]$, with corresponding commitments $(C_j, \hat{f}_j(\cdot)) \leftarrow \mathsf{PolyCom}(SP, f_j(\cdot))$ generated honestly. For any index i and any $j \in B$, an external verifier without access to the underlying shares $f_j(i)$ and $\hat{f}_j(i)$ can validate an evaluation proof via $\mathsf{PubVerEval}(SP, C_j, i, g^{f_j(i)} h^{\hat{f}_j(i)}, w_{j,i})$. Moreover, by invoking the aggregation interface $\mathsf{AggPubProEval}(SP, i, \{f_j(\cdot), \hat{f}_j(\cdot), C_j\}_{j\in B})$, the verifier can obtain the aggregated commitment–proof pair (T_i, W_i) and then execute $\mathsf{AggPubVerEval}(SP, i, T_i, W_i, \{C_j\}_{j\in B})$ to validate correctness, again relying solely on the public parameters and the commitments.

<u>*Evaluation binding:*</u> For any PPT adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{c} SP \leftarrow \mathsf{Setup}(1^\kappa, t); \\ \left(C, i, (f(i), \hat{f}(i), w_i), (f'(i), \hat{f}'(i), w'_i)\right) \leftarrow \mathcal{A}(SP) : \\ \mathsf{VerifyEval}(SP, C, i, f(i), \hat{f}(i), w_i) = 1 \wedge \\ \mathsf{VerifyEval}(SP, C, i, f'(i), \hat{f}'(i), w'_i) = 1 \wedge \\ f(i) \neq f'(i) \end{array}\right] \leq \varepsilon(\kappa)$$

Polynomial binding: For any PPT adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{c} SP \leftarrow \mathsf{Setup}(1^\kappa, t); \\ (I_1, I_2, C, \{f(i), \hat{f}(i), w_i\}_{i \in I_1 \cup I_2}) \leftarrow \mathcal{A}(SP) \\ \text{where } I_1, I_2 \subset [n], |I_1| = |I_2| = t+1: \\ \forall i \in I_1 \cup I_2, \\ \mathsf{VerifyEval}(SP, C, i, f(i), \hat{f}(i), w_i) = 1 \wedge \\ f_1(\cdot) \leftarrow \sum_{i \in I_1} \lambda_i \cdot f(i) \wedge \\ \hat{f}_1(\cdot) \leftarrow \sum_{i \in I_1} \lambda_i \cdot \hat{f}(i) \wedge \\ f_2(\cdot) \leftarrow \sum_{i \in I_2} \lambda_i \cdot f(i) \wedge \\ \hat{f}_2(\cdot) \leftarrow \sum_{i \in I_2} \lambda_i \cdot \hat{f}(i) \wedge f_1(\cdot) \neq f_2(\cdot) \end{array}\right] \leq \varepsilon(\kappa)$$

Aggregation binding: For any PPT adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{c} SP \leftarrow \mathsf{Setup}(1^\kappa, t); \\ (\{C_j\}_{j \in B}, i, (T_i, W_i, \{f_j(i), \hat{f}_j(i)\}_{j \in B}), \\ (\{f'_j(i), \hat{f}'_j(i), w'_{ji}\}_{j \in B})) \leftarrow \mathcal{A}(SP): \\ \mathsf{AggPubVerEval}(SP, i, T_i, W_i, \{C_j\}_{j \in B}) = 1 \wedge \\ \mathsf{AggOpenEval}(SP, i, T_i, \{f_j(i), \hat{f}_j(i), C_j\}_{j \in B}) = 1 \wedge \\ \forall j \in B, \mathsf{VerifyEval}(SP, C_j, i, f'_j(i), \hat{f}'_j(i), w'_{ji}) = 1 \wedge \\ \forall j \in B, f_j(i) \neq f'_j(i) \end{array}\right] \leq \varepsilon(\kappa)$$

Hiding: The commitments together with their aggregated evaluation proofs do not leak any information about the underlying polynomials beyond what is already revealed by the public parameters. This corresponds to the hiding property of the KZG scheme, extended to hold under aggregation. Formally, there exists PPT simulators $(\mathsf{Sim}_0, \mathsf{Sim}_1, \mathsf{Sim}_2, \mathsf{Sim}_3)$ such that for any PPT adversary $\mathcal{A}$, the following two distributions are computationally indistinguishable:

Real World:

$$\left\{\begin{array}{c} SP \leftarrow \mathsf{Setup}(1^\kappa, t); (\{f_j(\cdot)\}_{j \in B}, I_1, I_2) \leftarrow \mathcal{A}(SP) \\ \text{where } I_1, I_2 \subset [n], |I_1| \leq t, |I_2| \leq n; \\ \forall j \in B, (C_j, \hat{f}_j(\cdot)) \leftarrow \mathsf{PolyCom}(SP, f_j(\cdot)); \\ \forall i \in I_1, j \in B, \\ (f_j(i), \hat{f}_j(i), w_{ji}) \leftarrow \mathsf{ProveEval}(SP, i, f_j(\cdot), \hat{f}_j(\cdot)); \\ \forall i \in I_2, \\ (T_i, W_i) \leftarrow \mathsf{AggPubProEval}(SP, i, \{f_j(\cdot), \hat{f}_j(\cdot), C_j\}_{j \in B}); \\ : (C_j, \{i, f_j(i), \hat{f}_j(i), w_{ji}\}_{i \in I_1}, \{i, T_i, W_i\}_{i \in I_2}) \text{ where } j \in B \end{array}\right\}$$

Ideal World:

$$\left\{\begin{array}{c} (SP, st) \leftarrow \mathsf{Sim}_0(1^\kappa, t); (\{f_j(\cdot)\}_{j \in B}, I_1, I_2) \leftarrow \mathcal{A}(SP) \\ \text{where } I_1, I_2 \subset [n], |I_1| \leq t, |I_2| \leq n; \\ \forall j \in B, (C_j, c_j) \leftarrow \mathsf{Sim}_1(st); \\ \forall j \in B, i \in I_1, (w_{ji}, f'_j(\cdot), \hat{f}'_j(\cdot)) \leftarrow \mathsf{Sim}_2(st, C_j, c_j, i, f_j(i)); \\ \forall i \in I_2, (T_i, W_i) \leftarrow \mathsf{Sim}_3(SP, st, i, \{f'_j(\cdot), \hat{f}'_j(\cdot), C_j\}_{j \in B}) \\ : (C_j, \{i, f_j(i), \hat{f}_j(i), w_{ji}\}_{i \in I_1}, \{i, T_i, W_i\}_{i \in I_2}) \text{ where } j \in B \end{array}\right\}$$

We now state and prove the main security theorem.

Theorem A.1. *Assuming the hardness of the discrete logarithm problem and the t-strong Diffie–Hellman problem over the bilinear group, the augmented KZG scheme with aggregation interfaces satisfies correctness, public verifiability, evaluation binding, polynomial binding, aggregation binding, and hiding with overwhelming probability.*

Proof. We establish each property individually.

Correctness. The property holds by construction: any honestly generated proof, whether for a single evaluation or for an aggregation, always satisfies the corresponding verification equation.

Public Verifiability. The single-evaluation verification function $\mathsf{PubVerEval}(SP, C, i, g^{f(i)} h^{\hat{f}(i)}, w_i)$ as well as the aggregated verification function $\mathsf{AggPubVerEval}(SP, i, T_i, W_i, \{C_j\}_{j \in B})$ rely exclusively on public inputs and validate correctness through pairing checks. Since $(g^{f(i)} h^{\hat{f}(i)}, T_i, W_i)$ are Pedersen-style commitments, they inherit the hiding property, ensuring no additional information about evaluations or witnesses is leaked. Therefore, any external verifier can independently run PubVerEval or AggPubVerEval using only public transcripts, which suffices to establish public verifiability.

Evaluation Binding. Suppose there exists a PPT adversary $\mathcal{A}$ that breaks evaluation binding, i.e., it outputs two distinct tuples $(i, f(i), \hat{f}(i), w_i)$ and $(i, f'(i), \hat{f}'(i), w'_i)$ for the same evaluation point i, both of which are accepted by VerifyEval with non-negligible probability. Then an algorithm $\mathcal{B}$ can use $\mathcal{A}$ as a subroutine to break the t-SDH assumption with non-negligible probability. The reduction follows directly from the original KZG proof [38], and we omit the details here.

Polynomial Binding. Suppose that there exists a PPT adversary $\mathcal{A}$ that outputs a commitment C together with two distinct degree-t polynomials that both verify under VerifyEval with non-negligible probability. As shown in the original KZG proof [38], this would either imply a violation of evaluation binding or allow the extraction of two different polynomials committed to the same C, which directly yields an algorithm for solving the t-SDH or discrete logarithm assumption. Therefore, polynomial binding holds under these hardness assumptions.

Aggregation Binding. Suppose there exists a PPT adversary $\mathcal{A}$ that wins the aggregation binding game, i.e., it produces two distinct tuples for the same evaluation point i, namely $(T_i, W_i, \{f_j(i), \hat{f}_j(i), w_{ji}\}_{j \in B})$ and $(T'_i, W'_i, \{f'_j(i), \hat{f}'_j(i), w'_{ji}\}_{j \in B})$, where the evaluation proofs $\{w_{ji}\}_{j \in B}$ aggregate into W_i and the evaluation proofs $\{w'_{ji}\}_{j \in B}$ aggregate into W'_i under the same challenge γ. Assume further that both tuples are accepted by AggPubVerEval, AggOpenEval, and VerifyEval with non-negligible probability. We distinguish two cases:

- If $T_i \neq T'_i$, then two distinct openings are both accepted for the same evaluation point i, which directly contradicts the evaluation binding property of the KZG scheme.
- If $T_i = T'_i$, then

$$T_i = \prod_{j \in B} (g^{f_j(i)} h^{\hat{f}_j(i)})^{\gamma^{j-1}} = \prod_{j \in B} (g^{f'_j(i)} h^{\hat{f}'_j(i)})^{\gamma^{j-1}} = T'_i,$$

  which implies two different sets of values $\{f_j(i), \hat{f}_j(i)\}_{j \in B}$ and $\{f'_j(i), \hat{f}'_j(i)\}_{j \in B}$ produce the same Pedersen-style commitment. This contradicts the binding property of Pedersen commitments, which holds under the discrete logarithm assumption in $\mathbb{G}$.

Therefore, breaking the aggregation binding property would either break evaluation binding of the KZG scheme or the binding of Pedersen commitments, both of which are secure under standard hardness assumptions.

Hiding. We prove hiding by a sequence of games showing that the adversary's view is indistinguishable between the real protocol and the ideal world.

- **Game 0**: This is the real-world execution.
- **Game 1**: Same as Game 0, except that the trapdoors α and τ are sampled uniformly from $\mathbb{Z}_q$. Since the distribution of SP is identical in both games, Game 1 is indistinguishable from Game 0.
- **Game 2**: This is the ideal-world simulation. Here, the commitments, witnesses, and aggregation proofs are generated by the simulators $(\mathsf{Sim}_1, \mathsf{Sim}_2, \mathsf{Sim}_3)$ defined in Figure 10. All generated values pass verification against the commitments, hence the distribution of Game 2 is identical to that of Game 1.

Simulator for KZG with Aggregation

(1) $\mathsf{Sim}_0(1^\kappa, t)$: choose random trapdoors (α, τ), run $\mathsf{Setup}(1^\kappa, t)$ with them, and output (SP, st) where $st = (\alpha, \tau)$.
(2) $\mathsf{Sim}_1(st)$: for each $j \in B$, sample $c_j \xleftarrow{\$} \mathbb{Z}_q$ and set $C_j \leftarrow g^{c_j}$.
(3) $\mathsf{Sim}_2(st, C_j, c_j, \{i, f_j(i)\}_{i \in I_1})$: for $\forall j \in B$,
  (a) set $Q \leftarrow I_1$ and $Q' \leftarrow \emptyset$. Sample $i \in [n] \setminus I_1, f_j(i) \xleftarrow{\$} \mathbb{Z}_q$ and set $Q \leftarrow Q \cup \{i\}$ until $Q > t$.
  (b) interpolate $f'_j(\cdot)$ from $\{i, f_j(i)\}_{i \in Q}$.
  (c) compute $\hat{f}'_j(\alpha) \leftarrow \frac{c_j - f'_j(\alpha)}{\tau}$.
  (d) set $Q' \leftarrow \{\alpha\}$. Sample $k \in [n], \hat{f}'_j(k) \xleftarrow{\$} \mathbb{Z}_q$ and set $Q' \leftarrow Q' \cup \{k\}$ until $Q > t$.
  (e) interpolate $\hat{f}'_j(\cdot)$ from $\{k, f_j(k)\}_{k \in Q'}$.
  (f) for $\forall i \in I_1$, compute $w_{ji} \leftarrow g^{\frac{f'_j(\alpha) - f'_j(i)}{\alpha - i}} h^{\frac{\hat{f}'_j(\alpha) - \hat{f}'_j(i)}{\alpha - i}}$.
  (g) return $(\{w_{ji}\}_{i \in I_1}, f'_j(\cdot), \hat{f}'_j(\cdot))$.
(4) $\mathsf{Sim}_3(SP, st, i, \{f'_j(\cdot), \hat{f}'_j(\cdot), C_j\}_{j \in B})$: for $\forall i \in I_2$,
  (a) compute $\gamma \leftarrow \mathcal{H}(SP, i, \{C_j\}_{j \in B})$.
  (b) compute $T_i \leftarrow \prod_{j \in B} (g^{f'_j(i)} h^{\hat{f}'_j(i)})^{\gamma^{j-1}}$, and $W_i \leftarrow \prod_{j \in B} (g^{\frac{f'_j(\alpha) - f'_j(i)}{\alpha - i}} h^{\frac{\hat{f}'_j(\alpha) - \hat{f}'_j(i)}{\alpha - i}})^{\gamma^{j-1}}$.
  (c) return $(\{T_i, W_i\}_{i \in I_2})$.

Figure 10: Simulator $(\mathsf{Sim}_0, \mathsf{Sim}_1, \mathsf{Sim}_2, \mathsf{Sim}_3)$ for KZG polynomial commitment with aggregation.

□

B Security Analysis of AggTrans

Theorem B.1 (Termination, Transfer Consistency, and Secrecy of AggTrans). *If all honest servers in $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$ execute the* AggTrans*, then, with all but negligible probability, the following properties hold:*

- *Termination: All honest servers in $\mathcal{P}^{l+1}$ eventually obtain* `new_shares` *together with the corresponding randomness shares, evaluation proofs, and commitments.*
- *Transfer Consistency: For every gate computed in epoch C^l, if committee $\mathcal{P}^l$ holds a degree-t sharing $[s]^l$ encoding the secret s, then all honest servers in the next committee $\mathcal{P}^{l+1}$ reconstruct a fresh degree-t sharing $[s]^{l+1}$ encoding the same secret s.*
- *Secrecy: For the output* `new_shares`*, the adversary corrupting up to t servers in each of $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$ learns no additional information about $\{s_k\}_{k \in B}$ beyond what is revealed by the corrupted servers' own shares.*

Lemma B.2 (Termination). *All honest servers in $\mathcal{P}^{l+1}$ eventually obtain* `new_shares` *together with the corresponding randomness shares, evaluation proofs, and commitments.*

Proof. The AggTrans protocol involves two main communication components: BACSS and MVBA. By the completeness of BACSS, all honest servers in $\mathcal{P}^{l+1}$ will eventually receive valid shares from at least $2t+1$ dealers in $\mathcal{P}^l$. In addition, by the termination property of MVBA, all honest servers in $\mathcal{P}^{l+1}$ will eventually agree on the same common subset $T \subseteq \mathcal{P}^l$ of size $2t+1$. Once this subset T is fixed, each honest server P_j^{l+1} can locally interpolate its new shares $\{[s_k]_j^{l+1}, [\hat{s}_k]_j^{l+1}\}_{k \in [B]}$, and derive the corresponding evaluation proofs $\{w_{kj}^{l+1}\}_{k \in [B]}$ and commitments $\{C_k^{l+1}\}_{k \in [B]}$. Hence, all honest servers in $\mathcal{P}^{l+1}$ complete the protocol, thereby terminating the AggTrans.

□

Lemma B.3 (Transfer Consistency). *For every gate computed in epoch C^l, if committee $\mathcal{P}^l$ holds a degree-t sharing $[s]^l$ encoding the secret s, then all honest servers in the next committee $\mathcal{P}^{l+1}$ reconstruct a fresh degree-t sharing $[s]^{l+1}$ encoding the same secret s.*

Proof. By the completeness of BACSS, every honest server in $\mathcal{P}^{l+1}$ eventually receives valid shares lying on a valid degree-t polynomial. These shares collectively determine the aggregated constant term $T_{i,0}^{l+1}$, which in turn must satisfy two consistency requirements: on the one hand, it must match the constant terms of the newly generated commitments $\{C_{ki}^{l+1}\}_{k \in [B]}$, and on the other hand, it must coincide with the evaluations of the previous-epoch commitments $\{C_k^l\}_{k \in [B]}$ at the dealer's index i. This dual consistency guarantees that the values carried across epochs are preserved.

Finally, MVBA ensures that all honest servers agree on the same subset of dealers whose previous-epoch commitments $\{C_k^l\}_{k \in [B]}$ are identical. For this agreed-upon subset, each dealer's resharing polynomial is anchored to the same prior-epoch commitments, so that its constant term equals the dealer's previous-epoch share, i.e., a valid evaluation of the committed degree-t polynomial at index i. Based on this common subset, each honest server can interpolate new polynomials to obtain a fresh degree-t sharing in $\mathcal{P}^{l+1}$, which encodes the same secret as before. Thus, the outputs are preserved consistently across committees.

□

Lemma B.4 (Secrecy). *For the output* `new_shares`*, the adversary corrupting up to t servers in each of $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$ learns no additional information about $\{s_k\}_{k \in B}$ beyond what is revealed by the corrupted servers' own shares.*

Proof. We prove secrecy by constructing a sequence of game-based simulations, showing that a real-world adversary cannot

distinguish the actual protocol execution of AggTrans from a simulated one. Let $\mathcal{H}^l$ (resp. $\mathcal{H}^{l+1}$) be the honest servers in $\mathcal{P}^l$ (resp. $\mathcal{P}^{l+1}$), and let $\mathcal{A}^l$ (resp. $\mathcal{A}^{l+1}$) be the corrupted servers in each committee.

Game 0. The adversary corrupts up to t servers in $\mathcal{P}^l$ (forming $\mathcal{A}^l$) and up to t servers in $\mathcal{P}^{l+1}$ (forming $\mathcal{A}^{l+1}$). The honest servers $\mathcal{H}^l \cup \mathcal{H}^{l+1}$ execute the protocol exactly as prescribed by AggTrans. This represents the real-world protocol execution.

Game 1. The simulator first samples the common reference string (CRS) by choosing $\alpha, \tau \xleftarrow{\$} \mathbb{Z}_q$ and publishing $\{g, h = g^\tau, (g^\alpha, \ldots, g^{\alpha^t}), (h^\alpha, \ldots, h^{\alpha^t})\}$. It then impersonates all honest servers in $\mathcal{H}^l \cup \mathcal{H}^{l+1}$. For each honest dealer $P_i^l \in \mathcal{H}^l$, the simulator picks a random degree-t polynomial $f_i(\cdot)$ in $\mathbb{Z}_q[x]$ with $f_i(0) = [s]_i^l$. It uses $f_i(\cdot)$ as input to $\mathsf{BACSS_share}_i$ and invokes AggPubProEval in exactly the same way as honest servers in $\mathcal{H}^l$. The simulator also runs MVBA faithfully, ensuring that any corrupted server $P_j^{l+1} \in \mathcal{A}^{l+1}$ receives a set of shares $\{[[s]_i]_j^{l+1}\}_{i \in T}$ consistent with an agreed-upon subset $T \subseteq \mathcal{P}^l$, where $[[s]_i]_j^{l+1} = f_i(j)$.

Since the simulator follows the protocol specification exactly (from the viewpoint of any uncorrupted server), the adversary's view in **Game 1** is computationally indistinguishable from that in **Game 0**.

Game 2. Here, the simulator modifies how it provides shares to corrupted servers. Rather than genuinely invoking $\mathsf{BACSS_share}_i$ for each honest dealer P_i^l, the simulator takes as input the corrupted shares $\{[[s]_i^l]_j^{l+1}\}_{i \in \mathcal{H}^l, j \in \mathcal{A}^{l+1}}$ that the adversary would see. It then randomly samples a degree-t polynomial $f'(\cdot)$. For each $P_i^l \in \mathcal{H}^l$, the simulator interpolates a polynomial $f_i'(\cdot)$ consistent with these corrupted shares (so that $f_i'(j) = [[s]_i^l]_j^{l+1}$) and sets $f_i'(0) = f'(i)$ to fix any remaining degrees of freedom.

From the adversary's perspective, this change does not alter the distribution of the shares it observes. Moreover, the hiding property of the augmented KZG commitments ensures that, beyond the shares explicitly given to corrupted servers, no additional information about the underlying polynomials is revealed. Thus, the view of corrupted servers in **Game 2** remains indistinguishable from **Game 1**. Since **Game 1** is also indistinguishable from **Game 0**, it follows that AggTrans preserves secrecy against an adversary corrupting up to t parties in each of $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$.

□

C Security Analysis of BatchMul

Theorem C.1 (Termination, Multiplication Correctness, and Secrecy of BatchMul). *If all honest servers in $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$ execute the* BatchMul*, then, with all but negligible probability, the following properties hold:*

- *Termination: All honest servers in $\mathcal{P}^{l+1}$ eventually obtain* `mul_shares` *together with the corresponding randomness shares, evaluation proofs, and commitments.*
- *Multiplication Correctness: For each multiplication gate with inputs a and b, shared among $\mathcal{P}^l$ as $[a]^l$ and $[b]^l$, let $\langle c \rangle^l$ be the degree-$2t$ sharing of their product $c = a \cdot b$. Whenever $\mathcal{P}^l$ reshares $\langle c \rangle^l$, all honest servers in $\mathcal{P}^{l+1}$ reconstruct a new degree-t sharing $[c]^{l+1}$ whose constant term equals the correct product c.*
- *Secrecy: For the output* `mul_shares` $= \{[c_k]^{l+1}\}_{k \in [M]}$*, an adversary corrupting up to t servers in each of $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$ learns no information about the secrets $\{c_k\}_{k \in [M]}$ beyond what can be inferred from the shares held by the corrupted servers.*

Lemma C.2 (Termination). *All honest servers in $\mathcal{P}^{l+1}$ eventually obtain* `mul_shares` *together with the corresponding randomness shares, evaluation proofs, and commitments.*

Proof. In the BatchMul protocol, all n parallel BACSS instances are activated in parallel. By the completeness property of BACSS, every honest server in $\mathcal{P}^{l+1}$ will eventually receive valid shares from at least $2t+1$ dealers in $\mathcal{P}^l$. After performing the prescribed local verification to filter out invalid dealers, the servers in $\mathcal{P}^{l+1}$ invoke MVBA to agree on a common subset $T \subseteq \mathcal{P}^l$ of size $2t+1$. By the termination of MVBA, all honest servers in $\mathcal{P}^{l+1}$ eventually agree on the same common subset of dealers. Based on this subset, they interpolate to obtain the new `mul_shares` together with the corresponding randomness shares, evaluation proofs, and commitments, thereby completing the protocol. □

Lemma C.3 (Multiplication Correctness). *For each multiplication gate with inputs a and b, shared among $\mathcal{P}^l$ as $[a]^l$ and $[b]^l$, let $\langle c \rangle^l$ be the degree-$2t$ sharing of their product $c = a \cdot b$. Whenever $\mathcal{P}^l$ reshares $\langle c \rangle^l$, all honest servers in $\mathcal{P}^{l+1}$ reconstruct a new degree-t sharing $[c]^{l+1}$ whose constant term equals the correct product c.*

Proof. By the completeness of BACSS, every honest server in $\mathcal{P}^{l+1}$ eventually receives valid reshared values lying on degree-t polynomials. For each dealer P_i^l, verification is performed via $\mathsf{IPA}_{\mathsf{KZG}}$. In a single proof it certifies that $g^{\langle c_k \rangle_i^l} h^{\langle \hat{c}_k \rangle_i^l}$ equals the constant term of the newly published commitment $C_{c_k,i}^{l+1}$; that $g^{[a_k]_i^l} h^{[\hat{a}_k]_i^l}$ and $g^{[b_k]_i^l} h^{[\hat{b}_k]_i^l}$ correspond to the evaluations of the previous epoch commitments $C_{a_k}^l$ and $C_{b_k}^l$ at index i; and, crucially, via the inner product argument, that the committed values satisfy the multiplicative relation $c_k = a_k \cdot b_k$. These checks tie the reshared constant terms simultaneously to the old commitments and to the new commitment to c_k, ensuring that the product values are correct.

Finally, MVBA guarantees that all honest servers agree on the same subset $T \subseteq \mathcal{P}^l$ of size at least $2t+1$, such that the associated commitments $\{C_{a_k}^l, C_{b_k}^l\}_{k \in [M]}$ are identical. With the verified values from T, each honest server in $\mathcal{P}^{l+1}$ performs Lagrange interpolation at zero to reconstruct a fresh degree-t sharing, whose constant terms equal the correct product values.

□

Lemma C.4 (Secrecy). *For the output* `mul_shares` $= \{[c_k]^{l+1}\}_{k \in [M]}$*, an adversary corrupting up to t servers in each of $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$ learns no information about the secrets $\{c_k\}_{k \in [M]}$ beyond what can be inferred from the shares held by the corrupted servers.*

Proof. We establish secrecy via a sequence of games, showing that a real-world adversary cannot distinguish between the actual protocol execution and a simulated one. Let $\mathcal{H}^l$ and $\mathcal{H}^{l+1}$ be the

sets of honest servers in committees $\mathcal{P}^l$ and $\mathcal{P}^{l+1}$, respectively. Let $\mathcal{A}^l$ and $\mathcal{A}^{l+1}$ be the corresponding sets of corrupted servers.

Game 0. The adversary corrupts up to t servers in $\mathcal{P}^l$ (forming $\mathcal{A}^l$) and up to t servers in $\mathcal{P}^{l+1}$ (forming $\mathcal{A}^{l+1}$). The honest servers $\mathcal{H}^l \cup \mathcal{H}^{l+1}$ execute the protocol exactly as prescribed by BatchMul. This represents the real-world protocol execution.

Game 1. The simulator first samples the common reference string (CRS) by choosing $\alpha, \tau \xleftarrow{\$} \mathbb{Z}_q$ and publishing $\{g, h = g^\tau, (g^\alpha, \ldots, g^{\alpha^t}), (h^\alpha, \ldots, h^{\alpha^t})\}$. It then impersonates all honest servers in $\mathcal{H}^l \cup \mathcal{H}^{l+1}$. For each honest dealer $P_i^l \in \mathcal{H}^l$, the simulator samples a random degree-t polynomial $f_{ci}(\cdot)$ in $\mathbb{Z}_q$ with $f_{ci}(0) = \langle c \rangle_i^l$. It uses $f_{ci}(\cdot)$ as input to $\mathsf{BACSS_share}_i$ and invokes $\mathsf{IPA}_{\mathsf{KZG}}$ just as honest servers would. The simulator also runs MVBA faithfully, ensuring that any corrupted server $P_j^{l+1} \in \mathcal{A}^{l+1}$ receives a set of shares $\{[\langle c \rangle_i^l]_j^{l+1}\}_{i \in T}$ consistent with an agreed-upon subset $T \subseteq \mathcal{P}^l$, where $[\langle c \rangle_i^l]_j^{l+1} = f_{ci}(j)$.

Since the simulator follows the protocol specification exactly (from the viewpoint of any uncorrupted party), the adversary's view in **Game 1** is computationally indistinguishable from that in **Game 0**.

Game 2. Here, the simulator modifies how it provides shares to corrupted servers. Rather than genuinely invoking $\mathsf{BACSS_share}_i$ for each honest dealer P_i^l, the simulator takes as input the corrupted shares $\{[\langle c \rangle_i^l]_j^{l+1}\}_{i \in \mathcal{H}^l, j \in \mathcal{A}^{l+1}}$ that the adversary would see. It then randomly samples a degree-t polynomial $f_c'(\cdot)$. For each $P_i^l \in \mathcal{H}^l$, the simulator interpolates a polynomial $f_{ci}'(\cdot)$ consistent with these corrupted shares (so that $f_{ci}'(j) = [\langle c \rangle_i^l]_j^{l+1}$) and sets $f_{ci}'(0) = f_c'(i)$ to fix any remaining degrees of freedom.

From the adversary's perspective, this change does not alter the distribution of the shares it observes. The hiding property of the KZG with aggregation interfaces ensures that no additional information is revealed. Thus, the view of corrupted servers in **Game 2** remains indistinguishable from **Game 1**. Since **Game 1** is also indistinguishable from **Game 0**.

Game 3. This game is identical to **Game 2**, except that whenever an honest dealer P_i^l would run $\mathsf{IPA}_{\mathsf{KZG}}.\mathcal{P}$ to produce a proof for the public statement $(SP, \{C_{a_k}^l, C_{b_k}^l, C_{c_k}^{l+1}\}_{k \in [M]}, i_a, i_b, i_c, \mathcal{P}_{id})$, the simulator instead invokes the zero-knowledge simulator of the inner-product argument on the same public statement. By the perfect completeness of the inner-product argument, every simulated transcript for a true statement is accepted with non-negligible probability; by its statistical zero-knowledge property, the distribution of the simulated transcript is indistinguishable from that of a real transcript produced by $\mathsf{IPA}_{\mathsf{KZG}}.\mathcal{P}$. Hence the adversary's view in **Game 3** is indistinguishable from its view in **Game 2**.

Combining the indistinguishability of **Game 3** from **Game 2** with the previous transitions, we conclude that the adversary cannot distinguish the real execution **Game 0** from the simulated one, which completes the secrecy proof for BatchMul.

□

D Security and Complexity Analysis of AD-MPC++

Theorem D.1 (Privacy, and Guaranteed Output Delivery of Continuum). Continuum *securely realizes the desired properties of ADMPC (as in Definition 3.1).*

Proof. We prove that Continuum satisfies properties of ADMPC:

Privacy. During the input stage, secrecy is guaranteed by the underlying protocol BACSS. In the execution stage, secrecy is preserved separately for linear and multiplication gates: AggTrans (Lemma B.4) ensures that the evaluation of linear gates does not leak any information beyond the corrupted parties' shares, while BatchMul (Lemma C.4) guarantees the same for multiplication gates. In the output stage, clients broadcast shares to reconstruct the final outputs. Thus, throughout the entire execution, no additional information is leaked to an adversary beyond what could be inferred from the corrupted servers' own shares.

Guaranteed Output Delivery (GOD). During each epoch, circuit evaluation is divided into two parts. For linear gates, AggTrans ensures transfer consistency (Lemma B.3), i.e., the transferred shares reconstruct to the same underlying secrets, preserving correctness across committees. For multiplication gates, BatchMul guarantees multiplication correctness (Lemma C.3), ensuring that the multiplicative relations are faithfully preserved under resharing. Since both linear and multiplication gate evaluations remain correct within every epoch, the computation proceeds consistently across all epochs. Consequently, the final committee obtains correct output shares, which can be robustly reconstructed, and thus Continuum achieves guaranteed output delivery.

Combining these arguments, we conclude that Continuum satisfies privacy, and guaranteed output delivery as stipulated by Definition 3.1.

□

Complexity of Continuum. Consider an arithmetic circuit divided into D epochs, each of width W, consisting of λ linear gates and μ multiplication gates with $W = \lambda + \mu$. For each epoch, the communication cost is dominated by two sub-protocols: the asynchronous dynamic transfer of λ linear gates via AggTrans and the resharing of μ multiplication gates via BatchMul. The transfer of λ linear gates incurs a cost of $O(\kappa n^2 \cdot \lambda + \kappa n^3)$, while the resharing of μ multiplication gates requires $O(\kappa n^2 \cdot \mu + \kappa n^3)$. Summing these contributions, the total communication per epoch is

$$O(\kappa n^2 \cdot W + \kappa n^3).$$

Therefore, for a circuit of depth D, the overall communication complexity of Continuum is

$$O(\kappa n^2 \cdot WD + \kappa n^3 D).$$